SENIOR → STAFF DATA ENGINEER

Interview Question Bank
1,100 questions

100 questions for each of the 11 topics in the 15–week plan, from Python and SQL to system design, AI infrastructure and behavioral rounds. Questions only: practise answering out loud, then tick the box. At the back: 95 questions reported from real interviews, with sources.

1,100	11	15	+95
questions	topics	weeks	reported in real interviews

How to use it

- **Senior** questions are the ones to answer cold, in under 2 minutes.
- **Staff** questions go deeper: trade-offs, scale, and why you'd choose one option over another. Answer them the way you'd explain a decision to another team.
- Each section shows the week of the plan it belongs to. Do those questions alongside that week's work.

	TOPIC	QUESTIONS	SENIOR	STAFF
1	Python	PY-001 — PY-100	55	45
2	SQL	SQL-001 — SQL-100	56	44
3	DSA and coding	DSA-001 — DSA-100	57	43
4	Distributed systems and Kafka	DS-001 — DS-100	55	45
5	Spark	SP-001 — SP-100	55	45
6	Data modeling, warehousing and lakehouse	DM-001 — DM-100	52	48
7	AWS and Kubernetes	CL-001 — CL-100	52	48
8	ML, AI engineering and AI infrastructure	AI-001 — AI-100	52	48
9	Debugging and production scenarios	DB-001 — DB-100	52	48
10	System design	SD-001 — SD-100	35	65
11	Behavioral and leadership	BH-001 — BH-100	40	60
12	Reported in real interviews	RQ-01 — RQ-95	–	–

TOPIC 1 OF 11

Python

100 questions · 55 Senior · 45 Staff

Core language: data structures, comprehensions and typing Weeks 1–2

- [ ] PY-001 SENIOR What is the time complexity of a membership check on a list, a set and a dict, and when would you turn a 1 million item list into a set before a lookup loop?
- [ ] PY-002 STAFF How does a Python dict keep insertion order, and what happens to its memory when you delete half the keys from a 10 million key dict?
- [ ] PY-003 SENIOR When would you pick `collections.deque`, `defaultdict`, `Counter` or `OrderedDict` over a plain list or dict? Give one data pipeline use for each.
- [ ] PY-004 SENIOR Why must dict keys be hashable, and what goes wrong if you use a mutable object with a custom `__hash__` as a key and then change it?
- [ ] PY-005 SENIOR Rewrite a nested for loop that builds (user, event) pairs filtered by event type as a list comprehension, then explain when a comprehension hurts readability.
- [ ] PY-006 SENIOR How do a list comprehension, a generator expression and a dict comprehension differ in memory use when you process 50 million rows?
- [ ] PY-007 SENIOR Explain shallow copy versus deep copy using a nested config dict, and show the bug caused by using an empty list as a default argument.
- [ ] PY-008 SENIOR What is the difference between `is` and `==`, and why does `a is b` sometimes return True for small integers and short strings?
- [ ] PY-009 STAFF Compare tuples, `namedtuple`, `dataclass`, `dataclass(slots=True)` and `TypedDict` for holding 20 million records in memory, in terms of memory, speed and safety.
- [ ] PY-010 SENIOR How do type hints like `Optional`, `Union`, `Literal`, `TypedDict` and `Protocol` help in a pipeline codebase, given that Python does not enforce them at runtime?
- [ ] PY-011 STAFF How would you roll out `mypy` or `pyright` on a 200,000 line untyped pipeline repo without stopping feature work?
- [ ] PY-012 STAFF When would you use `TypeVar` and `Generic` in a data library, for example a typed `Reader` class that yields records of a given schema?

Functions, decorators, context managers, classes and exceptions Weeks 1–2

- [ ] PY-013 SENIOR Explain closures and late binding: why do lambdas created in a loop all return the last loop value, and how do you fix it?
- [ ] PY-014 SENIOR Write a decorator that logs a function's run time and arguments, and explain why you need `functools.wraps`.
- [ ] PY-015 SENIOR Write a decorator that takes arguments, such as `@retry(times=3)`, and explain why it needs three levels of nested functions.
- [ ] PY-016 STAFF How do decorators behave on instance methods, `classmethod`, `staticmethod` and async functions, and how would you write one decorator that works for both sync and async functions?

- [ ] PY-017 SENIOR Write a context manager that opens a database connection and always closes it, even when an exception is raised, once as a class with `__enter__` and `__exit__` and once with `contextlib.contextmanager`.
- [ ] PY-018 STAFF How would you use `contextlib.ExitStack` to manage a number of open files only known at runtime, for example merging 500 shard files at once, and what operating system limit might you hit?
- [ ] PY-019 SENIOR Explain `__init__`, `__repr__`, `__eq__` and `__hash__`, and what `@dataclass(frozen=True)` generates for you.
- [ ] PY-020 SENIOR What do `@property`, `@classmethod` and `@staticmethod` each do, and when would you use a classmethod as an alternate constructor like `from_json`?
- [ ] PY-021 STAFF When would you choose composition over inheritance for pipeline parts such as sources, transforms and sinks, and how do abstract base classes compare with `Protocol` for that design?
- [ ] PY-022 SENIOR Explain the order in which `try`, `except`, `else` and `finally` run, and what a function returns if both the `try` block and the `finally` block have a return statement.
- [ ] PY-023 SENIOR How would you design a custom exception hierarchy for a pipeline, for example retryable versus fatal errors, and why should you avoid a bare `except:`?
- [ ] PY-024 STAFF How would you use exception chaining with `raise ... from` and `ExceptionGroup` to report many bad records from one batch without losing the root cause?
- [ ] PY-025 STAFF Explain the descriptor protocol, how it powers `property`, and a case where a custom descriptor helps validate config fields.

Generators, iterators and memory Weeks 1–2

- [ ] PY-026 SENIOR Explain the iterator protocol (`__iter__` and `__next__`) and write a class that iterates over a file's lines in batches of 10,000.
- [ ] PY-027 SENIOR Write a generator that reads a 30 GB CSV line by line and yields parsed rows, and explain why memory stays flat.
- [ ] PY-028 SENIOR What happens when you loop over the same generator twice, and how does that cause silent bugs in code that counts rows and then processes them?
- [ ] PY-029 STAFF Build a lazy pipeline of generators (read → parse → filter → transform → write) and explain how back-pressure and error handling work when the writer is slow or one row fails.
- [ ] PY-030 SENIOR Use `itertools.groupby` to sum amounts per customer from a file sorted by customer, and explain why it gives wrong answers on unsorted input.
- [ ] PY-031 SENIOR How would you use `itertools.islice`, `chain`, `tee` and `batched` in a file processing job, and what is the memory risk with `tee`?
- [ ] PY-032 STAFF Explain `yield from`, `send()` and `close()` on generators, and when a coroutine-style generator is a reasonable design compared with a class.
- [ ] PY-033 STAFF Why does `sys.getsizeof` mislead you about the memory of a list of dicts, and how would you measure real memory use with `tracemalloc`?
- [ ] PY-034 STAFF A job that loads a 4 GB JSON file uses 30 GB of RAM. Why do Python objects inflate data this much, and what are three ways to cut it?

Performance, profiling and concurrency Weeks 1–2 and 13

- [ ] PY-035 SENIOR What is the GIL, which workloads does it slow down, and why does it not block threads that are waiting on I/O?
- [ ] PY-036 SENIOR Pick threading, multiprocessing or asyncio for each case and say why: downloading 10,000 files from S3, parsing 10,000 large JSON files, and calling an API with 500 concurrent requests.
- [ ] PY-037 SENIOR How do `ThreadPoolExecutor` and `ProcessPoolExecutor` from `concurrent.futures` differ, and what limits how much data you can pass to worker processes?
- [ ] PY-038 SENIOR Write an asyncio function that fetches 1,000 URLs with at most 50 in flight at once using a semaphore, with a timeout on each request.
- [ ] PY-039 STAFF What happens when you call a blocking library, such as a sync database driver, inside an asyncio event loop, and how do you find and fix it?
- [ ] PY-040 STAFF What does pickling cost when you send data between processes, and how can shared memory, memory-mapped files or Arrow buffers avoid the copies?
- [ ] PY-041 STAFF How does free-threaded Python (the no-GIL build from PEP 703) change your concurrency choices, and what would you check before using it in production?
- [ ] PY-042 SENIOR How do you profile a slow Python job with `cProfile` and `py-spy`, and how do you decide what to optimize first?
- [ ] PY-043 SENIOR Why is a vectorized NumPy or pandas operation often 50 times faster than a Python loop over the same 10 million rows?
- [ ] PY-044 STAFF A Python ETL job takes 6 hours on one machine. How would you decide between optimizing the code, parallelizing it, moving it to Polars or DuckDB, or moving it to Spark?
- [ ] PY-045 STAFF How would you find a memory leak in a long-running Python consumer whose memory grows by 200 MB every hour?
- [ ] PY-046 SENIOR Show a race condition on a shared counter in threaded Python code even with the GIL, and fix it with `threading.Lock`.

Data-manipulation coding prompts Weeks 1–2

- [ ] PY-047 SENIOR Given a list of dicts with `id`, `updated_at` and `payload`, keep only the latest record per `id`, breaking ties with a `sequence` number.
- [ ] PY-048 SENIOR Write a generator that takes click events sorted by user and time and yields each session as soon as it ends, where a new session starts after 30 minutes of inactivity.
- [ ] PY-049 SENIOR Given unsorted booking windows as (customer_id, start, end), merge overlapping or touching windows per customer and return total booked hours per customer.
- [ ] PY-050 SENIOR Write a function that flattens nested JSON (dicts inside lists inside dicts) into dot-separated column names, and decide what to do with arrays.
- [ ] PY-051 SENIOR Without pandas, group a list of order dicts by country and compute count, sum, average and max order value in one pass.
- [ ] PY-052 SENIOR Using `heapq`, return the 5 best-selling products per day from a stream of order events, and explain when `heapq.nlargest` is faster than sorting and when it is not.

- [ ] PY-053 SENIOR Implement an in-memory hash join of two lists of dicts, orders and customers, on `customer_id`, supporting inner and left joins, and say which side you build the hash table on.
- [ ] PY-054 STAFF Given the current dimension rows and a new daily snapshot as lists of dicts, write the SCD Type 2 logic that closes changed rows, opens new versions and handles keys that disappeared.
- [ ] PY-055 STAFF Write a point-in-time join that attaches to each training label the latest feature value at or before the label time, using `bisect` on sorted feature history, and explain how it prevents label leakage.
- [ ] PY-056 SENIOR Write a function that checks each row against a simple schema (required fields, types, allowed ranges) and returns good rows plus rejected rows with reasons.
- [ ] PY-057 SENIOR Parse a 5 GB application log where some entries span several lines (stack traces), and count errors per service per minute using only the standard library.
- [ ] PY-058 SENIOR Keep a sliding-window average over the last 5 minutes of sensor readings that arrive in time order, using a `deque`, with constant work per reading.
- [ ] PY-059 STAFF Compare two exports of a table by primary key and report added, removed and changed rows with the changed columns, first in memory for 2 million rows, then when neither file fits in memory.
- [ ] PY-060 STAFF Implement an external sort of a 50 GB file on a machine with 4 GB of RAM: write sorted runs to disk, then merge them with `heapq.merge`.
- [ ] PY-061 STAFF Deduplicate a stream of 1 billion event ids over a 24-hour window within 2 GB of memory; when would you accept a Bloom filter's false positives?
- [ ] PY-062 STAFF Given an ordered CDC log of insert, update and delete events where updates carry only the changed columns, rebuild the current state of every row.
- [ ] PY-063 SENIOR Write a deep merge for layered config dicts (base, then environment, then command line) where later layers win and lists are replaced, not appended.
- [ ] PY-064 SENIOR Convert timestamps that arrive in mixed formats and time zones to UTC, and explain the bugs from comparing naive and aware datetimes and from bucketing by local day across a daylight saving change.

Reliability and streaming patterns Weeks 3–4 and 9

- [ ] PY-065 SENIOR Write a retry decorator with exponential backoff and full jitter, a maximum number of attempts, and a list of exception types that are safe to retry.
- [ ] PY-066 STAFF Implement a thread-safe token-bucket rate limiter that keeps calls to an external API under 50 requests a second with bursts up to 100, and explain what breaks when 8 worker processes each use their own copy.
- [ ] PY-067 SENIOR Implement a cache for dimension lookups that combines LRU eviction with a 10-minute TTL per entry, and explain why `functools.lru_cache` alone is not enough.
- [ ] PY-068 STAFF Why can `functools.lru_cache` on an instance method keep objects alive and leak memory, and how does `functools.cache` differ from it?
- [ ] PY-069 SENIOR Write a client that pages through a REST API with cursor tokens, retries on 429 and 503 responses, and saves a checkpoint so a crashed run can resume.

- [ ] PY-070 STAFF How would you write a Python Kafka consumer that commits offsets only after a batch is written to the sink, and which failures lead to duplicates and which lead to data loss?
- [ ] PY-071 STAFF How would you make a Python batch job idempotent so it can be safely rerun for the same date partition after failing halfway through?
- [ ] PY-072 SENIOR How do you shut down a long-running Python worker gracefully when Kubernetes sends SIGTERM, so it finishes the current batch and commits progress before it is killed?
- [ ] PY-073 STAFF Design a circuit breaker for a pipeline that calls a flaky enrichment service, and explain its closed, open and half-open states and how you would test them.

pandas, Polars and PySpark APIs Weeks 5–6

- [ ] PY-074 SENIOR In pandas, how do `merge`, `join` and `concat` differ, and how do you find the rows that did not match after a left merge?
- [ ] PY-075 SENIOR Explain `groupby().agg()`, `transform()` and `apply()` in pandas, and why `apply` with a Python function is slow on 10 million rows.
- [ ] PY-076 SENIOR What causes `SettingWithCopyWarning`, and how does copy-on-write in recent pandas versions change the behavior?
- [ ] PY-077 SENIOR How would you shrink an 8 GB pandas DataFrame using dtypes such as `category`, smaller integer types and Arrow-backed strings?
- [ ] PY-078 STAFF When would you choose Polars or DuckDB over pandas for a 20 GB single-machine job, and what do lazy evaluation and query optimization give you?
- [ ] PY-079 SENIOR In PySpark, what is the difference between transformations and actions, and why does calling `count()` three times on the same DataFrame redo the work?
- [ ] PY-080 STAFF Compare a Python UDF, a pandas UDF that uses Arrow, and built-in Spark SQL functions on speed and serialization cost, and say when each one is acceptable.
- [ ] PY-081 SENIOR In PySpark, when do you use `repartition` versus `coalesce`, and what does `cache` versus `persist` with a storage level change?
- [ ] PY-082 STAFF Why are `collect()` and `toPandas()` on a large DataFrame dangerous, and how would you safely bring a 1% sample or the top 100 rows to the driver?
- [ ] PY-083 STAFF With the PySpark DataFrame API, read a nested JSON column, explode an array of line items and pivot item types into columns; what do `explode` and `pivot` cost at 1 billion rows?

Testing, packaging and code quality Weeks 1–2 and 14–15

- [ ] PY-084 SENIOR How do pytest fixtures, fixture scopes and `conftest.py` work, and how would you share one temporary test database across many tests?
- [ ] PY-085 SENIOR Use `pytest.mark.parametrize` to test a date parsing function against 15 edge cases, including leap days and time zone offsets.
- [ ] PY-086 SENIOR How do you mock a call to S3 or an HTTP API with `unittest.mock.patch`, and why does patching the wrong import path make the mock silently do nothing?
- [ ] PY-087 STAFF How would you use property-based testing with Hypothesis on a dedup or merge function, and what bugs does it find that example-based tests miss?

- [ ] PY-088 STAFF How would you unit test PySpark transformations with a shared local SparkSession fixture and small DataFrames so the whole suite runs in under 2 minutes?
- [ ] PY-089 SENIOR How do you structure a Python package with `pyproject.toml`, a src layout, pinned dependencies and a console script entry point?
- [ ] PY-090 STAFF Your org has 30 pipeline repos with drifting dependency versions; how would you standardize linting with ruff, type checks, lockfiles with uv or Poetry, and CI across them?

Python for ML and AI data pipelines Weeks 10–12

- [ ] PY-091 SENIOR Write a function that splits a dataset into train, validation and test sets by hashing the user id, so the split is deterministic and no user appears in two sets.
- [ ] PY-092 STAFF Write a token-aware text chunker that splits documents into chunks of at most 512 tokens with 64 tokens of overlap, keeping paragraph boundaries where possible.
- [ ] PY-093 STAFF Write an async batch client that sends 2 million prompts to an LLM API under rate limits, retries failures and writes results to disk so a crash loses at most one batch.
- [ ] PY-094 STAFF How would you find near-duplicate documents in a 10 million document training corpus with MinHash and locality-sensitive hashing in Python, and which thresholds would you tune?

Library and module design Weeks 14–15

- [ ] PY-095 STAFF Design an internal Python library that every team uses to read and write lakehouse tables: what would its public API, config, error types and versioning policy look like?
- [ ] PY-096 STAFF How would you design a plugin system so teams can add new sources (Postgres, S3, Kafka) to a shared ingestion framework without changing core code?
- [ ] PY-097 STAFF Design a data quality module (null rates, uniqueness, freshness, row count changes) that runs on both pandas and Spark; how do you avoid writing every check twice?
- [ ] PY-098 STAFF How would you design the config layer for a pipeline framework that reads YAML, environment variables and secrets, validates them with pydantic or dataclasses, and fails fast on bad values?
- [ ] PY-099 STAFF You own a shared Python library used by 40 teams; how do you ship a breaking change to a function signature without breaking their pipelines?
- [ ] PY-100 STAFF How would you build structured logging and metrics into a pipeline library so every job emits the same fields, such as job id, partition, row counts and duration?

TOPIC 2 OF 11

SQL

100 questions · 56 Senior · 44 Staff

Window functions: ranking, running totals and frames Weeks 1–2

- [ ] SQL-001 SENIOR Given `employees(employee_id, department, salary)`, show each employee with ROW_NUMBER, RANK and DENSE_RANK by salary within the department, and explain how the three differ on ties.
- [ ] SQL-002 SENIOR Given `orders(order_id, customer_id, order_date, amount)`, return every order with the customer's running total of amount by date, handling two orders on the same date consistently.
- [ ] SQL-003 SENIOR Given `orders(order_id, customer_id, order_date, amount)`, use LAG to return the days between each customer's consecutive orders, and the average gap per customer.
- [ ] SQL-004 SENIOR Given `stock_prices(ticker, trade_date, close_price)`, return each day's percent change from the previous trading day and flag days that dropped more than 5%.
- [ ] SQL-005 STAFF Explain ROWS BETWEEN versus RANGE BETWEEN, and show a query where the default frame with ORDER BY gives a wrong running total because of duplicate order keys.
- [ ] SQL-006 SENIOR Given `payments(user_id, paid_at, amount)`, return each user's first and last payment amount with FIRST_VALUE and LAST_VALUE, and explain why LAST_VALUE often returns the current row by mistake.
- [ ] SQL-007 SENIOR Given `sales(region, rep_id, revenue)`, return each rep's percent share of their region's total revenue using a window SUM instead of a join.
- [ ] SQL-008 SENIOR Given `scores(student_id, exam_date, score)`, show each score next to the student's previous and next score using LAG and LEAD, with a default value when there is none.
- [ ] SQL-009 SENIOR Given `products(product_id, category, price)`, split each category into 4 price quartiles with NTILE, and explain how NTILE splits a category that has 10 products.
- [ ] SQL-010 STAFF Given `events(user_id, event_ts, event_type)` with 5 billion rows, explain how a distributed engine runs a window function partitioned by user_id, and what happens when one user has 200 million events.
- [ ] SQL-011 STAFF Given `logins(user_id, login_ts)`, flag each login where the same user also logged in during the previous 7 days, once with a time-based RANGE frame and once with a self-join, and compare their cost.
- [ ] SQL-012 STAFF Given `account_ledger(account_id, txn_ts, delta)`, return the running balance per account and the first time each account's balance went below zero.

Top-N, dedup, self-joins and anti-joins Weeks 1–2

- [ ] SQL-013 SENIOR Given `employees(employee_id, department, salary)`, return the top 3 distinct salaries per department, including every employee who earns one of them.
- [ ] SQL-014 SENIOR Given `page_views(user_id, page_id, viewed_at)`, return the 2 most viewed pages per user, breaking ties by the most recent view.

- [ ] SQL-015 SENIOR Given `customer_updates(customer_id, email, status, updated_at, ingested_at)` with duplicates from retries, keep one row per customer: latest updated_at first, then latest ingested_at.
- [ ] SQL-016 STAFF Given `raw_events(event_id, user_id, event_ts, payload)` where the same event_id can arrive up to 3 days late, write an incremental dedup into a target table that scans only the last 3 days of the target, and explain the trade-off.
- [ ] SQL-017 SENIOR Given `users(user_id, signup_date)` and `orders(order_id, user_id, order_date)`, return users who never ordered using NOT EXISTS, and explain why NOT IN returns nothing when orders.user_id contains a NULL.
- [ ] SQL-018 SENIOR Given `employees(employee_id, name, manager_id, salary)`, return employees who earn more than their direct manager using a self-join.
- [ ] SQL-019 SENIOR Given `flights(flight_id, origin, destination)`, return every city pair that has flights in both directions, listing each pair only once.
- [ ] SQL-020 STAFF Given `purchases(user_id, product_id, purchase_date)`, return the 10 product pairs most often bought by the same user, and explain why the self-join explodes for users with 5,000 purchases and how you would cap it.
- [ ] SQL-021 SENIOR Given `customers(customer_id, email, created_at)`, find emails shared by more than one customer, treating case and surrounding spaces as equal, and list all customer ids per email.
- [ ] SQL-022 SENIOR Given `card_txns(txn_id, card_id, amount, txn_ts)`, find pairs of transactions on the same card with the same amount within 10 minutes of each other, as possible double charges.
- [ ] SQL-023 STAFF Given `users(user_id, country)` with 500 million rows and `banned_users(user_id)` with 2 million rows, compare LEFT JOIN with IS NULL, NOT EXISTS and EXCEPT for the anti-join, and explain which plan a distributed engine is likely to pick.

Gaps and islands and sessionization Weeks 1–2

- [ ] SQL-024 SENIOR Given `logins(user_id, login_date)` with possible duplicate rows per day, return users who logged in on 3 or more consecutive days.
- [ ] SQL-025 SENIOR Given `logins(user_id, login_date)`, return each user's longest streak of consecutive login days with its start and end dates.
- [ ] SQL-026 SENIOR Given `server_checks(server_id, check_ts, status)` sampled every minute, return each continuous period when a server was DOWN, with start, end and duration.
- [ ] SQL-027 SENIOR Given `seats(seat_id, is_free)`, return every run of 2 or more consecutive free seats with the first and last seat id.
- [ ] SQL-028 STAFF Given `driver_pings(driver_id, ping_ts, status)` sent every 30 seconds, compute total online minutes per driver per day, treating a gap of more than 2 minutes as offline and splitting periods that cross midnight.
- [ ] SQL-029 SENIOR Given `web_events(user_id, event_ts, page)`, assign a session number to each event where a new session starts after 20 minutes of inactivity, and return each session's start, end and page count.

- [ ] SQL-030 STAFF Extend that sessionization so a session also ends once it reaches 2 hours long, even with no gap, and explain why this rule is hard to express with window functions alone.
- [ ] SQL-031 SENIOR Given `invoices(invoice_number)`, return each gap in the invoice number sequence as a range of missing numbers, from first missing to last missing.
- [ ] SQL-032 STAFF Given `device_readings(device_id, reading_ts, temperature)` taken every minute, find periods where temperature stayed above 80 for at least 15 minutes in a row, tolerating one missing reading.

Retention, cohorts and funnels Weeks 1–2

- [ ] SQL-033 SENIOR Given `users(user_id, signup_date)` and `activity(user_id, activity_date)`, return day-1, day-7 and day-30 retention for each weekly signup cohort.
- [ ] SQL-034 SENIOR Build a monthly cohort retention table from the same tables: one row per signup month, one column for each of months 0 to 6 since signup, and values as the percent of the cohort active.
- [ ] SQL-035 SENIOR Given `events(user_id, event_ts, event_name)` with view_product, add_to_cart, checkout and purchase, count users who reach each step in order within 24 hours of their first view, and the step-to-step conversion.
- [ ] SQL-036 STAFF For that funnel, how do the numbers change if steps can happen in any order, or if you count sessions instead of users, and which version would you report to a product team and why?
- [ ] SQL-037 SENIOR Given `daily_activity(user_id, activity_date)`, label each user on each day as new, retained, resurrected or churned compared with the previous day, and count each label per day.
- [ ] SQL-038 SENIOR Given `activity(user_id, activity_date)`, compute DAU, WAU and MAU for every date in 2025, where WAU is the distinct users active in the trailing 7 days.
- [ ] SQL-039 STAFF Computing trailing 28-day distinct active users for every day over 3 billion activity rows takes 3 hours; how would you restructure it so each day does not rescan 28 days of raw data?
- [ ] SQL-040 STAFF Given `subscriptions(user_id, plan, start_date, cancel_date)`, compute monthly churn rate per plan, and defend your choice of denominator, such as subscribers active on the first day of the month.
- [ ] SQL-041 STAFF Given `orders(order_id, user_id, order_date, amount)`, compute 12-month customer lifetime value per acquisition cohort, and explain how you handle cohorts younger than 12 months.
- [ ] SQL-042 STAFF Given `ab_assignments(user_id, variant, assigned_at)` and `purchases(user_id, purchased_at, amount)`, compute conversion rate and revenue per user per variant, counting only purchases after assignment, and say how you would handle users assigned to both variants.

Time series: growth, rolling windows and date spines Weeks 1–2

- [ ] SQL-043 SENIOR Given `monthly_revenue(month, product_line, revenue)`, return year-over-year growth percent per product line and month, handling months where last year's revenue was zero or missing.

- [ ] SQL-044 SENIOR Given `daily_sales(sale_date, store_id, amount)` with missing days, return a 7-day rolling average per store that treats missing days as zero sales.
- [ ] SQL-045 SENIOR Use a recursive CTE or a series generator to build a date spine for every day from 2024-01-01 to 2025-12-31, then left join `signups(user_id, signup_date)` so days with no signups show 0.
- [ ] SQL-046 SENIOR Given `daily_sales(sale_date, amount)`, return month-to-date and year-to-date totals for every day.
- [ ] SQL-047 SENIOR Given `weekly_metrics(week_start, value)`, return the week-over-week change and flag weeks more than 3 standard deviations away from the trailing 8-week mean.
- [ ] SQL-048 SENIOR Given `daily_metrics(metric_date, revenue)`, compare each day with the same weekday 52 weeks earlier instead of the same calendar date, and explain why that is often the better comparison.
- [ ] SQL-049 STAFF Given `inventory_changes(product_id, change_date, quantity)` written only when quantity changes, return every product's quantity on every day of the quarter (forward fill) without a huge cross join.
- [ ] SQL-050 STAFF Given `orders(order_id, order_ts, amount, store_timezone)` stored in UTC for stores in 12 time zones, return daily revenue by each store's local business day, where the business day ends at 4 a.m. local time.
- [ ] SQL-051 STAFF Over `events(event_ts, value)` with 2 billion rows, compare a 30-day rolling sum done with a RANGE INTERVAL window against pre-aggregating to daily totals first, and explain which one a warehouse runs faster and why.

Medians, percentiles, pivots and recursive CTEs Weeks 1-2

- [ ] SQL-052 SENIOR Given `deliveries(order_id, city, delivery_minutes)`, return the median delivery time per city, once with PERCENTILE_CONT and once without any percentile function.
- [ ] SQL-053 SENIOR Given `requests(endpoint, latency_ms, request_ts)`, return p50, p95 and p99 latency per endpoint per hour, and explain PERCENTILE_CONT versus PERCENTILE_DISC.
- [ ] SQL-054 STAFF Exact p99 over 10 billion latency rows is slow; explain at a high level how APPROX_PERCENTILE or t-digest sketches work, their error, and whether you can merge hourly sketches into a daily p99.
- [ ] SQL-055 SENIOR Given `quarterly_sales(product_id, quarter, revenue)` with quarters Q1 to Q4, pivot to one row per product and one column per quarter using conditional aggregation.
- [ ] SQL-056 SENIOR Given `product_stats(product_id, views, clicks, purchases)`, unpivot it into (product_id, metric_name, metric_value) rows, once with UNPIVOT and once with UNION ALL or a CROSS JOIN to a VALUES list.
- [ ] SQL-057 SENIOR Given `employees(employee_id, name, manager_id)`, use a recursive CTE to return each employee's level below the CEO and their full management chain as one string.
- [ ] SQL-058 STAFF Given `bill_of_materials(parent_part, child_part, quantity)`, return the total quantity of every base part needed to build one unit of part X, and guard the recursion against cycles.
- [ ] SQL-059 SENIOR Given `categories(category_id, parent_id, name)` and `products(product_id, category_id)`, count products under each category including all of its subcategories.

- [ ] SQL-060 SENIOR Given `user_tags(user_id, tag)`, return one row per user with all tags as a sorted comma-separated string, then split that string back into rows.
- [ ] SQL-061 STAFF For a hierarchy of 5 million nodes that changes daily, when would you query it with a recursive CTE at read time versus maintaining a precomputed closure table, and what does each cost?

Slowly changing dimensions and modeling queries Week 7

- [ ] SQL-062 SENIOR Given `dim_customer(customer_key, customer_id, tier, valid_from, valid_to, is_current)` as SCD Type 2, return every customer's tier as of 2025-03-15.
- [ ] SQL-063 SENIOR Given that dimension and `fact_orders(order_id, customer_id, order_ts, amount)`, join each order to the tier valid at order time, and explain inclusive versus exclusive valid_to and how the open-ended current row is stored.
- [ ] SQL-064 STAFF Write a MERGE that applies today's `stg_customer(customer_id, tier, email)` to the SCD2 dimension, expiring changed rows and inserting new versions in one statement, and explain why plain MERGE needs a trick to do both.
- [ ] SQL-065 SENIOR Given the SCD2 dimension, return customers whose tier changed more than twice in 2025, with the ordered list of tiers they moved through.
- [ ] SQL-066 STAFF Given `customer_snapshots(snapshot_date, customer_id, tier, email)` with one full copy per day for 3 years, build SCD2 rows with valid_from and valid_to, collapsing days where nothing changed.
- [ ] SQL-067 SENIOR Write data quality queries on an SCD2 table that find overlapping validity periods, gaps between versions, and customers with more than one current row.
- [ ] SQL-068 SENIOR Given `fact_sales(date_key, product_key, store_key, amount)` with date and product dimensions, roll up sales by month and category with subtotals, and explain when GROUPING SETS, ROLLUP or CUBE replace several queries.
- [ ] SQL-069 STAFF Given `fact_balance(account_id, snapshot_date, balance)` as a daily periodic snapshot, explain why summing balance across days is wrong, and return average daily balance and end-of-month balance per month.
- [ ] SQL-070 SENIOR Given `order_items(order_id, product_id, quantity, price)` and `orders(order_id, customer_id, shipping_fee)`, compute revenue per customer without double counting the order-level shipping fee.
- [ ] SQL-071 STAFF Given `account_owners(account_id, customer_id, ownership_pct)` as a bridge between `accounts(account_id, balance)` and customers, return balance per customer and explain how to avoid inflated totals.

Lakehouse, incremental and streaming SQL Weeks 4 and 8

- [ ] SQL-072 SENIOR Given a target table partitioned by event_date and a staging table of late events spread over 6 dates, write an incremental load that overwrites only the affected partitions, and explain static versus dynamic partition overwrite.
- [ ] SQL-073 STAFF Explain what MERGE INTO does at the file level on an Iceberg or Delta table with copy-on-write versus merge-on-read, and why a MERGE that touches 1% of rows can rewrite 60% of the files.

- [ ] SQL-074 SENIOR Use Delta or Iceberg time travel to compare a table's row count and total revenue now versus 24 hours ago, to debug a bad overnight load.
- [ ] SQL-075 STAFF Given `cdc_batch(id, op, changed_at, name, status)` from Debezium, where op is c, u or d and one id can appear several times, write a MERGE that applies the batch to the target and explain why you must reduce the batch to one row per id first.
- [ ] SQL-076 STAFF In Flink SQL or Spark Structured Streaming, write a 1-minute tumbling window count of events by country, and explain how the watermark decides when a window is final and what happens to an event that is 10 minutes late.
- [ ] SQL-077 STAFF Explain how a streaming SQL join of impressions to clicks within 1 hour bounds its state, and what you lose if you set the time bound too small or too large.

SQL engine internals and performance

Weeks 3, 5–6 and 13

- [ ] SQL-078 SENIOR Read the EXPLAIN plan of a slow query that joins orders to customers: how do you spot a full scan, the join type, and the step where row counts explode?
- [ ] SQL-079 STAFF Compare an index in Postgres, partitioning in a warehouse, and clustering or Z-ordering in a lakehouse, and give a query each one speeds up and the write cost each one adds.
- [ ] SQL-080 SENIOR When does a B-tree index help a query, and when will the planner ignore it, for example with a function on the column, a LIKE with a leading wildcard, or low selectivity?
- [ ] SQL-081 STAFF Compare broadcast hash join, shuffle hash join and sort-merge join: when does each win, and how do table sizes like 50 MB versus 5 TB drive the choice?
- [ ] SQL-082 STAFF A join on customer_id is slow because 30% of rows have customer_id = 0 for guest checkouts; show two SQL rewrites that fix the skew, such as handling the hot key separately or salting.
- [ ] SQL-083 SENIOR Explain three-valued logic: what do NULL = NULL, COUNT(col) versus COUNT(1), SUM and AVG over NULL values, and GROUP BY on a NULL key return?
- [ ] SQL-084 SENIOR What is the difference between WHERE and HAVING, and between putting a filter in the ON clause versus the WHERE clause of a LEFT JOIN?
- [ ] SQL-085 STAFF Why is COUNT(DISTINCT user_id) over 20 billion rows expensive in a distributed engine, and how do APPROX_COUNT_DISTINCT and HyperLogLog trade about 1 to 2% error for speed and memory?
- [ ] SQL-086 SENIOR Why is selecting all columns slow and costly on a columnar warehouse like BigQuery or Snowflake, and how do column pruning and partition pruning cut the bytes scanned?
- [ ] SQL-087 SENIOR What is predicate pushdown, and give an example where a cast or function in the WHERE clause stops partition pruning on a date-partitioned table.
- [ ] SQL-088 STAFF A query that ran in 2 minutes yesterday takes 40 minutes today with no code change; walk through how you find out why (data volume, skew, stale stats, plan change, spill, concurrency).
- [ ] SQL-089 SENIOR When should you use UNION versus UNION ALL, and DISTINCT versus GROUP BY, and what does each cost on a billion-row table?
- [ ] SQL-090 STAFF How does a cost-based optimizer choose join order, why do stale table statistics cause bad plans, and when would you add a join hint?

- [ ] SQL-091 SENIOR Rewrite a correlated subquery that finds each customer's latest order date as a join or a window function, and explain why the correlated version can be slow.
- [ ] SQL-092 STAFF How do materialized views, result caching and pre-aggregated summary tables differ, and which would you use for a dashboard queried 10,000 times a day on data that refreshes hourly?
- [ ] SQL-093 STAFF Compare a row-oriented OLTP database and a columnar MPP warehouse running the same aggregation, covering storage layout, vectorized execution and compression.

Staff-level metrics and reconciliation Weeks 13–15

- [ ] SQL-094 STAFF Product asks for daily active users; write the exact definition you would propose (what counts as active, time zone, bots, logged-out users, deleted accounts) and the SQL for it over `events(user_id, device_id, event_ts, event_name, is_bot)`.
- [ ] SQL-095 STAFF The finance dashboard shows 10.2 million dollars of March revenue and the product dashboard shows 9.7 million; list the queries you would run to find the gap, such as refunds, currency, time zones, late data, filters and duplicates.
- [ ] SQL-096 STAFF Two teams report different conversion rates from the same tables; how do you find each team's definition, and how would you make one definition the source of truth?
- [ ] SQL-097 STAFF Given `orders(order_id, order_ts, amount, currency)` and `fx_rates(currency, rate_date, usd_rate)`, return daily revenue in USD, and explain which rate you pick and what happens on days with no rate.
- [ ] SQL-098 STAFF A key metric jumped 15% overnight; write the SQL checks you would run to decide whether it is a real change or a pipeline bug, such as row counts by source, null rates, duplicate keys and distribution shifts.
- [ ] SQL-099 STAFF Write SQL that shows Simpson's paradox, where overall conversion falls while conversion in every country rises, and explain how you would present it to leadership.
- [ ] SQL-100 STAFF Show in SQL the difference between an average of ratios and a ratio of totals, for example average order value per user versus total revenue divided by total orders, and say which one a metric spec should use.

TOPIC 3 OF 11

DSA and coding

100 questions · 57 Senior · 43 Staff

Arrays, hashing and two pointers Weeks 1–2

- [ ] DSA-001 SENIOR Two Sum (common in DE loops)
- [ ] DSA-002 SENIOR Group Anagrams (common in DE loops)
- [ ] DSA-003 SENIOR Product of Array Except Self
- [ ] DSA-004 SENIOR Longest Consecutive Sequence (common in DE loops)
- [ ] DSA-005 SENIOR Subarray Sum Equals K (common in DE loops)
- [ ] DSA-006 SENIOR Insert Delete GetRandom O(1)
- [ ] DSA-007 STAFF Encode and Decode Strings, then compare your format with the length-prefixed encoding used by Protobuf or Avro
- [ ] DSA-008 STAFF First Missing Positive in O(n) time and O(1) extra space, and explain why the in-place index trick works
- [ ] DSA-009 STAFF Majority Element, then extend it to every value that appears more than n/3 times and explain how this connects to heavy-hitter counting on streams
- [ ] DSA-010 SENIOR 3Sum (common in DE loops)
- [ ] DSA-011 SENIOR Container With Most Water
- [ ] DSA-012 STAFF Trapping Rain Water with two pointers, and explain why it beats the prefix-max arrays version on memory
- [ ] DSA-013 SENIOR Merge Sorted Array (common in DE loops)
- [ ] DSA-014 STAFF Sort Colors in one pass, and explain how the same three-way partition idea splits records into three buckets without extra memory

Sliding window, stacks and binary search Weeks 1–2

- [ ] DSA-015 SENIOR Best Time to Buy and Sell Stock (common in DE loops)
- [ ] DSA-016 SENIOR Longest Substring Without Repeating Characters (common in DE loops)
- [ ] DSA-017 SENIOR Longest Repeating Character Replacement
- [ ] DSA-018 STAFF Minimum Window Substring, and explain why the total work is linear even though the left pointer moves inside an inner loop
- [ ] DSA-019 STAFF Sliding Window Maximum with a monotonic deque, and explain why each element is pushed and popped at most once
- [ ] DSA-020 SENIOR Valid Parentheses (common in DE loops)
- [ ] DSA-021 SENIOR Min Stack
- [ ] DSA-022 SENIOR Daily Temperatures

- [ ] DSA-023 SENIOR Evaluate Reverse Polish Notation
- [ ] DSA-024 STAFF Largest Rectangle in Histogram, and explain the invariant the monotonic stack keeps
- [ ] DSA-025 SENIOR Search in Rotated Sorted Array
- [ ] DSA-026 SENIOR Find First and Last Position of Element in Sorted Array (common in DE loops)
- [ ] DSA-027 STAFF Koko Eating Bananas, then use the same binary search on the answer to find the smallest number of workers that finishes a batch of tasks within 4 hours
- [ ] DSA-028 STAFF Median of Two Sorted Arrays, and explain why you binary search on the smaller array

Linked lists, heaps and intervals Weeks 1–2

- [ ] DSA-029 SENIOR Reverse Linked List (common in DE loops)
- [ ] DSA-030 SENIOR Merge Two Sorted Lists (common in DE loops)
- [ ] DSA-031 SENIOR LRU Cache (common in DE loops)
- [ ] DSA-032 STAFF LFU Cache with O(1) get and put, and explain when LFU beats LRU for a lookup cache
- [ ] DSA-033 SENIOR Kth Largest Element in an Array, solved with a heap and with quickselect, comparing their worst cases (common in DE loops)
- [ ] DSA-034 STAFF Top K Frequent Elements, solved with a heap and with bucket sort, and explain which you pick when k is close to n (common in DE loops)
- [ ] DSA-035 STAFF Find Median from Data Stream, and explain why two heaps beat keeping a sorted list (common in DE loops)
- [ ] DSA-036 STAFF Smallest Range Covering Elements from K Lists
- [ ] DSA-037 SENIOR Task Scheduler
- [ ] DSA-038 SENIOR Insert Interval (common in DE loops)
- [ ] DSA-039 SENIOR Meeting Rooms II (common in DE loops)
- [ ] DSA-040 SENIOR Non-overlapping Intervals
- [ ] DSA-041 SENIOR Interval List Intersections
- [ ] DSA-042 STAFF Employee Free Time

Trees and tries Weeks 1–2

- [ ] DSA-043 SENIOR Binary Tree Level Order Traversal (common in DE loops)
- [ ] DSA-044 SENIOR Validate Binary Search Tree
- [ ] DSA-045 SENIOR Lowest Common Ancestor of a Binary Tree
- [ ] DSA-046 SENIOR Kth Smallest Element in a BST
- [ ] DSA-047 SENIOR Diameter of Binary Tree
- [ ] DSA-048 STAFF Construct Binary Tree from Preorder and Inorder Traversal

- [ ] DSA-049 STAFF Serialize and Deserialize Binary Tree, and compare your format with how Parquet stores nested data using repetition and definition levels
- [ ] DSA-050 STAFF Binary Tree Maximum Path Sum
- [ ] DSA-051 SENIOR Implement Trie (Prefix Tree)
- [ ] DSA-052 SENIOR Design Add and Search Words Data Structure
- [ ] DSA-053 STAFF Word Search II, and explain why a trie with pruning beats searching for each word separately
- [ ] DSA-054 STAFF Design Search Autocomplete System, then explain how you would keep the top 3 suggestions per prefix fresh with 10,000 new queries a second

Graphs: BFS, DFS, topological sort, union-find and Dijkstra Weeks 1–2

- [ ] DSA-055 SENIOR Number of Islands (common in DE loops)
- [ ] DSA-056 SENIOR Clone Graph
- [ ] DSA-057 SENIOR Rotting Oranges
- [ ] DSA-058 SENIOR Course Schedule (common in DE loops)
- [ ] DSA-059 SENIOR Course Schedule II (common in DE loops)
- [ ] DSA-060 STAFF Alien Dictionary
- [ ] DSA-061 SENIOR Number of Connected Components in an Undirected Graph, using union-find with path compression and union by rank
- [ ] DSA-062 SENIOR Redundant Connection
- [ ] DSA-063 SENIOR Accounts Merge (common in DE loops)
- [ ] DSA-064 SENIOR Network Delay Time
- [ ] DSA-065 STAFF Cheapest Flights Within K Stops, and explain why plain Dijkstra on cost alone gives wrong answers here
- [ ] DSA-066 STAFF Word Ladder, and explain how bidirectional BFS cuts the search
- [ ] DSA-067 STAFF Evaluate Division, and connect it to converting amounts between currencies from a table of exchange rates

Dynamic programming and greedy Weeks 1–2

- [ ] DSA-068 SENIOR House Robber
- [ ] DSA-069 SENIOR Coin Change (common in DE loops)
- [ ] DSA-070 STAFF Longest Increasing Subsequence, first in O(n squared) and then in O(n log n), and explain why the patience-sorting array works
- [ ] DSA-071 SENIOR Longest Common Subsequence
- [ ] DSA-072 STAFF Edit Distance, then explain why you cannot run it on every pair of 10 million company names in a dedup job and what blocking strategy you would use instead

- [ ] DSA-073 SENIOR Word Break
- [ ] DSA-074 SENIOR Decode Ways
- [ ] DSA-075 STAFF Partition Equal Subset Sum, and relate it to splitting 200 files of different sizes into two equal-sized batches
- [ ] DSA-076 STAFF Regular Expression Matching
- [ ] DSA-077 SENIOR Maximum Subarray (common in DE loops)
- [ ] DSA-078 SENIOR Jump Game
- [ ] DSA-079 SENIOR Gas Station
- [ ] DSA-080 SENIOR Partition Labels

Data-engineering-flavored coding Weeks 1-2, 3 and 4

- [ ] DSA-081 SENIOR Merge k sorted log files, each sorted by timestamp, into one sorted stream without loading any file fully into memory (common in DE loops)
- [ ] DSA-082 SENIOR Find the top 10 URLs in a 100 GB access log on a machine with 8 GB of RAM (common in DE loops)
- [ ] DSA-083 SENIOR Time Based Key-Value Store (common in DE loops)
- [ ] DSA-084 SENIOR Implement a sort-merge join of two large files already sorted by key, handling duplicate keys on both sides (common in DE loops)
- [ ] DSA-085 SENIOR Write a CSV line parser that handles quoted fields, doubled quotes inside quoted fields, and commas inside quotes, without using the csv module (common in DE loops)
- [ ] DSA-086 STAFF Implement a consistent-hash ring with virtual nodes that supports add_node, remove_node and get_node for a key, and measure how many keys move when you add an eleventh node to 10
- [ ] DSA-087 STAFF Design an in-memory rate limiter that allows 100 requests per user per minute using a sliding window counter, and compare its memory with a sliding window log for 10 million users
- [ ] DSA-088 STAFF Implement an in-memory key-value store with get, set, delete, begin, commit and rollback, where transactions can be nested
- [ ] DSA-089 STAFF Given a DAG of pipeline tasks with run times, compute the earliest time the whole pipeline can finish with unlimited workers, and return the critical path
- [ ] DSA-090 STAFF Keep a uniform random sample of 1,000 events from a stream of unknown length with reservoir sampling, and show why every event has the same chance of being kept
- [ ] DSA-091 STAFF Events arrive up to 5 minutes out of order; re-emit them in timestamp order with bounded memory using a min-heap, and decide what to do with events that are later than 5 minutes
- [ ] DSA-092 STAFF Implement a Snowflake-style ID generator (timestamp, machine id, sequence) that produces roughly time-sorted 64-bit ids on 1,000 machines without coordination, and handle the clock moving backwards

Follow-up at scale Weeks 3, 5–6 and 14–15

- [ ] DSA-093 STAFF Group Anagrams follow-up: the input is now 200 GB of words in 100 files on S3; what changes, and how would you do it with Spark or MapReduce?
- [ ] DSA-094 STAFF Kth Largest Element in an Array follow-up: 10 billion numbers now sit on 1,000 machines; how do you find the exact k-th largest with few network round trips?
- [ ] DSA-095 STAFF Accounts Merge follow-up: there are now 3 billion email-to-account links; how would you find connected components in a distributed engine, and how many rounds could it take?
- [ ] DSA-096 STAFF Course Schedule follow-up: the DAG now has 50,000 pipeline tasks and changes every hour; how do you detect cycles on each change and run ready tasks in parallel?
- [ ] DSA-097 STAFF LRU Cache follow-up: 64 threads now share it and it holds 50 million entries; what changes, and how does sharding cut lock contention?
- [ ] DSA-098 STAFF Time Based Key-Value Store follow-up: it must now survive restarts and hold 1 TB on disk; what storage layout would you pick, and how does a read find the right version?
- [ ] DSA-099 STAFF Subarray Sum Equals K follow-up: the array is now a 200 GB file split into 400 chunks processed in parallel; how do you combine the per-chunk results correctly?
- [ ] DSA-100 STAFF Merge k sorted log files follow-up: the logs now total 5 TB on 500 machines and you need one globally sorted output in S3; how do you split the work so no single machine merges everything?

TOPIC 4 OF 11

Distributed systems and Kafka

100 questions · 55 Senior · 45 Staff

Replication and consistency Week 3

- [ ] DS-001 SENIOR Explain how single-leader replication works, and what happens to the last 200 acknowledged writes if the leader crashes before any follower has copied them.
- [ ] DS-002 SENIOR What is the difference between synchronous and asynchronous replication, and why do many databases run one synchronous follower and the rest asynchronous?
- [ ] DS-003 SENIOR A user updates their profile, refreshes the page, and sees the old value because the read hit a lagging replica. Which guarantee is broken, and what are two ways to fix it?
- [ ] DS-004 SENIOR Define monotonic reads and consistent prefix reads, and give one concrete anomaly that each one prevents.
- [ ] DS-005 SENIOR In a leaderless store with N = 3, explain why W + R greater than N makes read and write quorums overlap, and compare W = 2, R = 2 with W = 1, R = 1.
- [ ] DS-006 STAFF Even with W + R greater than N, a leaderless store can return stale or lost data. Walk through at least three ways this happens, such as sloppy quorums, concurrent writes, and a partly failed write that was never rolled back.
- [ ] DS-007 SENIOR What are read repair, hinted handoff and anti-entropy with Merkle trees, and which problem does each one solve in a Dynamo-style store?
- [ ] DS-008 STAFF You need writes accepted in three regions with under 50 ms local latency. Compare multi-leader replication using last-write-wins, CRDTs, and application-level merge, first for a shopping cart and then for an account balance.
- [ ] DS-009 SENIOR What is the difference between linearizability and serializability, and can a system provide one without the other?
- [ ] DS-010 STAFF Why is linearizability expensive across regions, and which parts of a data platform truly need it, such as leader election, unique name claims or job locks?
- [ ] DS-011 SENIOR Explain eventual consistency in plain words, and what "eventual" means in practice when replica lag is usually 100 ms but spikes to 30 seconds during a backlog.
- [ ] DS-012 STAFF Explain causal consistency, why it is often described as the strongest model that can stay available during a network partition, and one product feature where it is enough.
- [ ] DS-013 SENIOR State the CAP theorem precisely, and explain why "pick two out of three" is a misleading summary of it.
- [ ] DS-014 STAFF Use PACELC to classify a single-leader Postgres setup with async replicas, a Dynamo-style store, and a Spanner-style database, and say what each one gives up when there is no partition.

Partitioning, consensus, clocks and failure detection Week 3

- [ ] DS-015 SENIOR Compare range partitioning and hash partitioning for data keyed by `user_id` and `event_time`, including which queries each one makes fast or slow.

- [ ] DS-016 SENIOR Explain consistent hashing with virtual nodes, and roughly what fraction of keys move when you add a 6th node to a 5-node cluster, compared with plain hash mod N.
- [ ] DS-017 STAFF Compare three rebalancing strategies: a fixed large number of partitions, dynamic splitting by size, and a fixed number of partitions per node. Which would you choose for a store growing from 2 TB to 200 TB, and why?
- [ ] DS-018 SENIOR Compare local (document-partitioned) and global (term-partitioned) secondary indexes, and the read and write cost of each for a query that filters on `country`.
- [ ] DS-019 SENIOR Explain how Raft elects a leader, including terms, randomized election timeouts, and why a candidate needs votes from a majority.
- [ ] DS-020 STAFF In Raft, when is a log entry committed, and why can a new leader not commit entries from an older term just by counting how many nodes have them?
- [ ] DS-021 SENIOR Why do consensus clusters usually have 3 or 5 members instead of 4 or 6, and how many failures can each size survive?
- [ ] DS-022 STAFF At a concept level, what problem does Paxos solve, what do proposers and acceptors do, and why do most teams build on etcd, ZooKeeper or a proven Raft library instead of writing consensus code themselves?
- [ ] DS-023 STAFF What does the FLP impossibility result say, and how do real systems such as Raft still make progress in practice?
- [ ] DS-024 SENIOR Why can you not order events from different servers by their wall-clock timestamps, and what is the difference between a time-of-day clock and a monotonic clock?
- [ ] DS-025 SENIOR Explain Lamport timestamps, and show why they give an order consistent with causality but cannot tell you whether two events were concurrent.
- [ ] DS-026 STAFF How do vector clocks detect concurrent writes, why does their size grow with the number of writers, and what do hybrid logical clocks offer as a middle ground?
- [ ] DS-027 STAFF Spanner's TrueTime exposes clock uncertainty as an interval, and the database waits it out before a commit becomes visible. Explain commit wait and why tighter clock bounds directly cut write latency.
- [ ] DS-028 SENIOR How does a heartbeat-and-timeout failure detector work, what goes wrong with a 1-second timeout versus a 60-second timeout, and how does a phi accrual detector improve on a fixed timeout?

Delivery guarantees, idempotency and backpressure Week 3

- [ ] DS-029 SENIOR Define at-most-once, at-least-once and exactly-once delivery, and explain why exactly-once is really "effectively once" processing rather than a property of the network.
- [ ] DS-030 SENIOR Design an idempotent payment-capture API using idempotency keys. Where do you store the keys, how long do you keep them, and what do you return when a client retries?
- [ ] DS-031 SENIOR Why should retries use exponential backoff with jitter, and what happens to a recovering service when 10,000 clients all retry every second in lockstep?
- [ ] DS-032 STAFF A worker reads a message from a queue, writes a row to a database, then acknowledges the message. Walk through each crash point, the loss or duplicate it causes, and how you make the whole flow effectively exactly-once.

- [ ] DS-033 SENIOR What is a split-brain or zombie leader, and how do fencing tokens with an increasing number stop an old leader from corrupting shared storage?
- [ ] DS-034 STAFF Why is a lease-based lock with a 10-second TTL unsafe on its own when a process can pause for 15 seconds in garbage collection, and how would you make it safe?
- [ ] DS-035 SENIOR What is backpressure? Compare buffering, dropping or sampling, and slowing the producer when a source sends 500,000 events a second into a consumer that can handle 200,000.
- [ ] DS-036 STAFF Explain how a pull-based consumer like Kafka's and a credit-based flow control design like Flink's network stack each create backpressure, and where the waiting data ends up in each case.
- [ ] DS-037 STAFF Compare two-phase commit with the saga pattern for an order flow that touches inventory, payments and shipping, including what happens if the coordinator dies in the middle.
- [ ] DS-038 SENIOR What is the transactional outbox pattern, and which dual-write problem does it solve when a service must update its database and also publish an event?

Kafka core: brokers, replication, producers and retention Week 4

- [ ] DS-039 SENIOR Explain how a Kafka topic, partition, log segment and offset relate to each other, and what happens on disk when a producer appends a record.
- [ ] DS-040 SENIOR What do replication factor, leader replica, follower replica and the in-sync replica set (ISR) mean, and what causes a follower to drop out of the ISR?
- [ ] DS-041 SENIOR Compare `acks=0`, `acks=1` and `acks=all`, and describe the exact failure that loses an acknowledged write under `acks=1`.
- [ ] DS-042 SENIOR With replication factor 3, `min.insync.replicas=2` and `acks=all`, what do producers see when one broker is down, and when two brokers are down?
- [ ] DS-043 STAFF What does `unclean.leader.election.enable` trade off, and on which kinds of topics would you allow it and never allow it?
- [ ] DS-044 SENIOR How does the idempotent producer prevent duplicates using a producer ID and sequence numbers, and why can it keep ordering with up to 5 in-flight requests per connection?
- [ ] DS-045 SENIOR How does Kafka's default partitioner pick a partition for a record with a key, and what does it do with records that have a null key in recent versions?
- [ ] DS-046 STAFF You grow a topic from 12 to 48 partitions. What happens to the key-to-partition mapping and per-key ordering, and how would you migrate safely if consumers depend on ordering per `account_id`?
- [ ] DS-047 SENIOR How do `linger.ms`, `batch.size` and `compression.type` affect producer throughput and latency, and what would you try first for 300 MB a second of small JSON events?
- [ ] DS-048 STAFF Walk through a Kafka transaction end to end: `transactional.id`, the transaction coordinator, producer epochs, commit markers, and what a consumer with `isolation.level=read_committed` sees.
- [ ] DS-049 STAFF Kafka exactly-once covers read-process-write inside Kafka. Explain exactly where that guarantee stops when the sink is Postgres or S3, and what you must add to close the gap.

- [ ] DS-050 SENIOR Compare `cleanup.policy=delete` and `cleanup.policy=compact`, and give a real use case for each.
- [ ] DS-051 SENIOR In a compacted topic, how do tombstones work, what does `delete.retention.ms` control, and why can a consumer still see two values for the same key?
- [ ] DS-052 SENIOR A topic has `retention.ms` set to 7 days and also a `retention.bytes` limit. How do the two limits interact, why is data deleted in whole segments, and how does segment size affect when old data actually disappears?
- [ ] DS-053 STAFF Kafka replaced ZooKeeper with KRaft. What does the KRaft controller quorum do, and which operational problems does it remove for a cluster with 200,000 partitions?
- [ ] DS-054 STAFF When does tiered storage for Kafka make sense, and how does it change retention, broker disk sizing and broker recovery time for a 90-day replayable topic that takes in 2 TB a day?

Kafka consumers, ordering and operations Week 4

- [ ] DS-055 SENIOR How does a consumer group assign partitions to its members, and what happens if the group has 20 consumers but the topic has 12 partitions?
- [ ] DS-056 SENIOR Compare auto-commit, synchronous commit after processing, and asynchronous commit of offsets, including the duplicate or loss risk of each.
- [ ] DS-057 SENIOR What triggers a consumer group rebalance, and what role do `session.timeout.ms`, `heartbeat.interval.ms` and `max.poll.interval.ms` each play?
- [ ] DS-058 STAFF Compare eager rebalancing, cooperative incremental rebalancing, static membership with `group.instance.id`, and the newer broker-driven consumer group protocol. Which would you use for 300 consumers on Kubernetes with rolling deploys?
- [ ] DS-059 SENIOR What ordering guarantee does Kafka give, and why does handing records from one partition to a thread pool break it?
- [ ] DS-060 STAFF You need per-customer ordering for 50 million customers at 400,000 events a second. How do you choose the partition key and partition count, and plan for three years of growth?
- [ ] DS-061 STAFF One partition gets 15 times more traffic than the rest because a few tenants are huge. Compare key salting, a separate topic for large tenants, and a custom partitioner, including what each does to ordering.
- [ ] DS-062 SENIOR What is consumer lag, how do you measure it in offsets and in time, and why is lag in time usually the better alert?
- [ ] DS-063 SENIOR How do you replay the last 6 hours of a topic for one consumer group without affecting other groups reading the same topic?
- [ ] DS-064 SENIOR What is a poison message, and how do you design retry topics and a dead-letter topic so one bad record does not block a partition?
- [ ] DS-065 STAFF Size a Kafka cluster for 1 GB a second of ingress, replication factor 3, 7 days of retention and 5 consumer groups. Walk through the disk, network and partition math.
- [ ] DS-066 STAFF Compare one large shared multi-tenant Kafka cluster with many smaller clusters per domain, covering blast radius, quotas, cost and upgrades.

Schemas, Kafka Connect and CDC Week 4

- [ ] DS-067 SENIOR What does a schema registry do, how does a producer serializer register or look up a schema ID, and where is that ID carried in each message?
- [ ] DS-068 SENIOR Explain BACKWARD, FORWARD and FULL compatibility, and which of them allow adding a field with a default or removing a field.
- [ ] DS-069 STAFF Your registry enforces BACKWARD compatibility, but a team wants to rename a field read by 30 downstream consumers. Lay out a safe migration plan, and explain why the transitive compatibility modes matter.
- [ ] DS-070 SENIOR Compare Avro, Protobuf and JSON Schema for Kafka event payloads, covering message size, evolution rules and tooling.
- [ ] DS-071 SENIOR Explain the parts of Kafka Connect: workers, connectors, tasks, converters and single message transforms. What runs where in distributed mode?
- [ ] DS-072 STAFF An S3 sink connector writes 40,000 tiny files an hour. Which settings and design changes reduce the file count, and what do they cost in latency and delivery guarantees?
- [ ] DS-073 SENIOR How does a log-based CDC tool like Debezium read changes from MySQL or Postgres, and why is it better than polling an `updated_at` column?
- [ ] DS-074 STAFF A Debezium Postgres connector was down for 2 days and the database disk is filling up. Explain why, in terms of replication slots and WAL retention, and how you would recover and prevent it next time.
- [ ] DS-075 SENIOR Describe the Debezium change event fields (`before`, `after`, `op`, `source`), what the tombstone after a delete is for, and how the initial snapshot works before streaming begins.
- [ ] DS-076 STAFF You need to stream changes from a 3 TB MySQL table into a lakehouse without long locks, and later re-snapshot one table after a bug. Explain your approach, including incremental snapshots and keeping order per primary key.

Stream processing with Flink, Spark Structured Streaming and Kafka Streams Week 4

- [ ] DS-077 SENIOR Explain event time versus processing time, and what a watermark tells a stream processor.
- [ ] DS-078 SENIOR Compare tumbling, sliding (hopping) and session windows, with a concrete metric that fits each one.
- [ ] DS-079 STAFF Events from mobile apps can arrive up to 2 hours late after a device was offline. Compare a long watermark delay, allowed lateness with updated results, and a side output for late events plus a batch correction job.
- [ ] DS-080 SENIOR At a concept level, how do Flink checkpoints work with barriers and asynchronous state snapshots, and how is a checkpoint different from a savepoint?
- [ ] DS-081 STAFF How does Flink get end-to-end exactly-once into Kafka with a two-phase commit sink, and why must the Kafka transaction timeout cover the checkpoint interval plus recovery time while staying within the broker's `transaction.max.timeout.ms`?
- [ ] DS-082 SENIOR What is keyed state in a stream processor, and why do jobs with large state use a RocksDB state backend instead of keeping state on the JVM heap?
- [ ] DS-083 STAFF A Flink job has 4 TB of keyed state and each checkpoint takes 9 minutes. What would you look at, and how can incremental checkpoints, state TTL and key design help?

- [ ] DS-084 SENIOR In Kafka Streams, what is the difference between a KStream, a KTable and a GlobalKTable, and when do you need each one for a join?
- [ ] DS-085 SENIOR How does Kafka Streams make local state fault tolerant with changelog topics and standby replicas, and what happens during state restore after a pod restarts?
- [ ] DS-086 STAFF Why do Kafka Streams joins need co-partitioned inputs, and what do the resulting repartition topics cost in latency, storage and broker load?
- [ ] DS-087 STAFF Pick Flink, Spark Structured Streaming or Kafka Streams for each of three cases: sub-second fraud scoring, 5-minute aggregates written to a lakehouse table, and a small enrichment microservice. Explain each choice.
- [ ] DS-088 STAFF How would you join a stream of 200,000 clicks a second with a stream of ad impressions where matching events can be up to 30 minutes apart, and how do you keep the join state bounded?

Debugging streaming systems Week 13

- [ ] DS-089 SENIOR Consumer lag on one partition keeps growing while the other 47 partitions are fine. List what you would check, in order.
- [ ] DS-090 SENIOR A consumer group rebalances every few minutes and throughput falls close to zero. What are the likely causes, and which logs and metrics confirm each one?
- [ ] DS-091 STAFF After a broker restart, producers get `NotEnoughReplicasException` and under-replicated partitions stay high for an hour. How do you diagnose and fix it?
- [ ] DS-092 SENIOR A downstream table shows duplicate events after a consumer deploy. How do you tell whether they came from producer retries, consumer reprocessing or the sink, and how do you clean them up?
- [ ] DS-093 STAFF A Flink job's watermark stopped advancing and windows never fire. What can cause this, such as an idle partition, and how do you fix it?
- [ ] DS-094 STAFF End-to-end latency from app event to dashboard jumped from 5 seconds to 4 minutes, but no component reports errors. How do you measure latency stage by stage to find the bottleneck?

Staff-level streaming design judgment Weeks 14–15

- [ ] DS-095 STAFF Design a real-time pipeline that takes 1 million events a second from mobile clients and feeds both a fraud model and an analytics lakehouse. Cover ingestion, partitioning, schemas, processing, sinks and failure handling.
- [ ] DS-096 STAFF How would you design a self-service streaming platform for 40 product teams so they can create topics, register schemas and get quotas safely without every team needing Kafka experts?
- [ ] DS-097 STAFF Your company runs self-managed Kafka on EC2 and spends two engineers' time on operations. How would you evaluate moving to Amazon MSK or Confluent Cloud, and what would you measure before and after?
- [ ] DS-098 STAFF A stakeholder wants exactly-once for every pipeline. How do you explain what it really costs, and when would you accept at-least-once with idempotent sinks instead?
- [ ] DS-099 STAFF Design a multi-region Kafka setup for disaster recovery with a 5-minute RPO, including topic mirroring, consumer offset translation, and how failover and failback work.

☐ DS-100 STAFF A bad schema rollout broke 12 consumers for 3 hours. Run the post-incident review: which guardrails, runbooks and review steps would you add so it cannot happen the same way again?

TOPIC 5 OF 11

Spark

100 questions · 55 Senior · 45 Staff

Execution model and lazy evaluation Weeks 5–6

- [ ] SP-001 SENIOR Explain the roles of the driver, the executors and the cluster manager, and what runs where when you call `df.groupBy("country").count().show()`.
- [ ] SP-002 SENIOR How does Spark turn one action into jobs, stages and tasks, and what decides where one stage ends and the next one starts?
- [ ] SP-003 SENIOR What is lazy evaluation in Spark, and what are two concrete benefits it gives the optimizer?
- [ ] SP-004 SENIOR Sort these into transformations and actions, and say which ones start a job: `filter`, `count`, `withColumn`, `write`, `collect`, `cache`, `show`.
- [ ] SP-005 SENIOR Explain narrow versus wide dependencies with two examples of each, and why wide dependencies make recovery from failure more expensive.
- [ ] SP-006 SENIOR What is lineage in Spark, and how does Spark rebuild a lost partition when an executor dies in the middle of a job?
- [ ] SP-007 SENIOR What changes between client and cluster deploy mode in `spark-submit`, and when does the choice matter in production?
- [ ] SP-008 SENIOR What is the difference between RDDs, DataFrames and Datasets, and why do most PySpark production pipelines use DataFrames?
- [ ] SP-009 STAFF A job adds 300 columns one at a time with `withColumn` in a loop, and planning becomes very slow or fails with a stack overflow. Explain why, and how you would fix it.
- [ ] SP-010 STAFF What is speculative execution, and when does turning on `spark.speculation` hurt more than it helps?
- [ ] SP-011 SENIOR What happens when a task fails 4 times (`spark.task.maxFailures`), and how is a stage retry after a fetch failure different from a task retry?
- [ ] SP-012 STAFF How does dynamic allocation decide when to add executors and when to release idle ones, and how would you tune it for a cluster shared by 2-minute and 6-hour jobs?

Shuffles, partitions and partition sizing Weeks 5–6

- [ ] SP-013 SENIOR What physically happens during a shuffle, from map-side writes to local disk to reduce-side fetches, and why is it usually the most expensive step in a job?
- [ ] SP-014 SENIOR What does `spark.sql.shuffle.partitions` control, what is its default, and why is that default wrong for both a 2 GB job and a 20 TB job?
- [ ] SP-015 SENIOR How does Spark decide how many input partitions to create when reading 400 GB of Parquet, and what do `spark.sql.files.maxPartitionBytes` and `spark.sql.files.openCostInBytes` change?
- [ ] SP-016 SENIOR Compare `repartition(n)`, `repartition(n, col)` and `coalesce(n)`, and explain why `coalesce(1)` at the end of a big job can squeeze the whole final stage onto one core.

- [ ] SP-017 STAFF What size do you aim for per shuffle partition and why, and how would you pick the partition count for a job that shuffles 6 TB on 400 cores?
- [ ] SP-018 SENIOR Which of these cause a shuffle: `groupBy`, `join`, `distinct`, `orderBy`, a window with `partitionBy`, `union`, `mapPartitions`? Explain each answer.
- [ ] SP-019 STAFF Why is a global `orderBy` on a huge dataset expensive, including the sampling step for range partitioning, and when is `sortWithinPartitions` enough?
- [ ] SP-020 STAFF A job repartitions by `customer_id` and then writes partitioned by `event_date`, producing thousands of files per date. Explain why, and how you would reshape the data before the write.
- [ ] SP-021 SENIOR Explain map-side partial aggregation, and why `reduceByKey` beats `groupByKey` on RDDs for computing a sum per key.
- [ ] SP-022 STAFF What problems do push-based shuffle and remote shuffle services such as Apache Celeborn solve at very large scale, and what do they cost to run?

Catalyst, Tungsten, AQE and UDFs Weeks 5–6

- [ ] SP-023 SENIOR Name the phases of the Catalyst optimizer from unresolved logical plan to selected physical plan, and give one example of a rule applied during logical optimization.
- [ ] SP-024 SENIOR What does Project Tungsten do for Spark performance, including whole-stage code generation and the compact binary row format?
- [ ] SP-025 SENIOR In an `explain("formatted")` output, what do `Exchange`, `BroadcastExchange`, `HashAggregate`, `SortMergeJoin`, and a `FileScan` with `PushedFilters` each tell you?
- [ ] SP-026 STAFF A plan shows a predicate under `PushedFilters`, yet the job still reads far more data than expected. Why can that happen, and how do you check whether row groups or files were actually skipped?
- [ ] SP-027 SENIOR What are the three main features of Adaptive Query Execution, and why can AQE only change the plan at stage boundaries?
- [ ] SP-028 STAFF AQE coalesced 2,000 shuffle partitions into 40 and the job got slower. How can that happen, and which settings control the coalescing?
- [ ] SP-029 SENIOR What does the cost-based optimizer need to work well, and how do stale or missing table statistics lead to bad join choices?
- [ ] SP-030 SENIOR Why are row-at-a-time Python UDFs slower than built-in functions? Explain how data moves between the JVM and the Python worker processes.
- [ ] SP-031 SENIOR How do pandas UDFs using Apache Arrow reduce that overhead, and what kinds of logic still justify a pandas UDF instead of a built-in function?
- [ ] SP-032 STAFF A filter applies a Python UDF to a partition column, and partition pruning and predicate pushdown stop working. Explain why, and how you would rewrite the filter.
- [ ] SP-033 STAFF When would you write a Scala UDF or a native Spark expression instead of a pandas UDF, and how would you decide for a function that runs 30 billion times a day?
- [ ] SP-034 STAFF How does dynamic partition pruning work when a partitioned fact table is joined to a filtered date dimension, and what conditions must hold for it to kick in?

Joins and skew Weeks 5–6

- [ ] SP-035 SENIOR Explain how a broadcast hash join, a shuffle hash join and a sort-merge join work, and when Spark chooses each by default.
- [ ] SP-036 SENIOR What does `spark.sql.autoBroadcastJoinThreshold` do, what is its default, and what can go wrong if you raise it to 2 GB?
- [ ] SP-037 SENIOR In a left outer join, which side can Spark broadcast, and why can a full outer join not use a broadcast hash join?
- [ ] SP-038 SENIOR How do you force a join strategy with the `BROADCAST`, `MERGE` and `SHUFFLE_HASH` hints, and when is overriding the optimizer a bad idea?
- [ ] SP-039 SENIOR Skew can come from the data itself, from a poor partitioning function, or from placeholder values like `'unknown'` or `0`. Give an example of each and how you would detect it.
- [ ] SP-040 STAFF Explain key salting for a skewed join step by step: how you salt the large side, replicate the small side, and choose the salt factor.
- [ ] SP-041 STAFF How does AQE's skew join optimization split skewed partitions, which settings decide what counts as skewed, and which join types or cases it cannot fix?
- [ ] SP-042 STAFF In a left join, 30% of the left side has a null join key and one task runs for hours. Why do the nulls pile up in one task, and what are your options?
- [ ] SP-043 SENIOR What are a cartesian product and a broadcast nested loop join, and how can a non-equi condition such as a date range accidentally produce one?
- [ ] SP-044 STAFF You join a 20 TB events table with a 300 GB users table every day. Compare a plain sort-merge join, bucketing both tables on `user_id`, and a pre-joined layout, including total cost over a year.
- [ ] SP-045 SENIOR How does skew show up in aggregations rather than joins, and how does a two-stage aggregation with a salted key help?
- [ ] SP-046 STAFF A range join matches 2 billion IP events against 5 million IP ranges and runs for 10 hours. How would you redesign it?

Memory, caching, spills and OOM Weeks 5–6

- [ ] SP-047 SENIOR Explain Spark's unified memory model: execution memory, storage memory, user memory and reserved memory, and what `spark.memory.fraction` controls.
- [ ] SP-048 SENIOR What is executor memory overhead, what lives there (for example Python worker processes and off-heap buffers), and why do PySpark executors get killed for exceeding container memory on YARN or Kubernetes?
- [ ] SP-049 SENIOR What does spilling mean in Spark, what is the difference between "spill (memory)" and "spill (disk)" in task metrics, and is spilling always a problem?
- [ ] SP-050 SENIOR List four common causes of executor OOM, such as one huge partition, an exploding join, or too many cores per executor, and give a fix for each.
- [ ] SP-051 STAFF An executor with 16 GB of heap and 4 cores dies with OOM while processing a 3 GB shuffle partition. Walk through the per-task memory math and three different ways to fix it.
- [ ] SP-052 SENIOR When should you cache a DataFrame, which storage level does `df.cache()` use by default, and why should you call `unpersist()` afterwards?

- [ ] SP-053 STAFF A team caches every intermediate DataFrame "for speed", and their jobs now run slower with more spills and evictions. Explain why, and what rule of thumb you would give them.
- [ ] SP-054 STAFF Executors spend 30% of task time in garbage collection. How do you confirm it, which JVM and Spark settings would you try, and when is the real fix fewer cores per executor or a different data layout?
- [ ] SP-055 SENIOR Why do `collect()` and `toPandas()` on a large result crash the driver, what does `spark.driver.maxResultSize` protect against, and what should you use instead?
- [ ] SP-056 STAFF An `explode` on an array column turns 1 billion rows into 80 billion and executors start dying. How do you restructure the job to keep memory under control?

File formats, small files and data layout Weeks 5–6

- [ ] SP-057 SENIOR Compare Parquet, ORC and Avro as storage formats on layout, compression, schema evolution and the workloads each fits best.
- [ ] SP-058 SENIOR Explain Parquet's internal structure: row groups, column chunks, pages, and the footer with min and max statistics.
- [ ] SP-059 SENIOR Explain the difference between partition pruning and predicate pushdown into the file reader, and give a query that benefits from one but not the other.
- [ ] SP-060 SENIOR Why can a filter such as `CAST(user_id AS STRING) = '42'` or `lower(country) = 'us'` stop predicate pushdown, and how would you write each filter so pushdown still works?
- [ ] SP-061 STAFF How would you choose the partition column for a 500 TB table that is queried mostly by date and sometimes by `tenant_id`, and what goes wrong if you partition by `user_id`?
- [ ] SP-062 STAFF Why are small files a problem for Spark on object storage, and how would you fix a table with 2 million files averaging 300 KB each?
- [ ] SP-063 STAFF How do sorting rows within files and Z-ordering improve data skipping, and what do they cost at write time?
- [ ] SP-064 SENIOR How does the `maxRecordsPerFile` write option help control output file size, and why can it not fix files that are too small?
- [ ] SP-065 STAFF Why do Spark bucketed tables often fail to remove shuffles in practice, for example because of mismatched bucket counts, Hive versus Spark bucketing, or many files per bucket?
- [ ] SP-066 SENIOR What are Parquet dictionary encoding and Parquet bloom filters, and for which column types and queries does each one help?
- [ ] SP-067 SENIOR How does schema merging work when reading Parquet files written with different schemas, and why is `mergeSchema` off by default?
- [ ] SP-068 STAFF You inherit 3 PB of gzip-compressed JSON on S3. Make the case for converting it to Parquet with Snappy or Zstd, including the one-time conversion cost and the savings you would expect.

Structured Streaming and lakehouse writes Weeks 5–6

- [ ] SP-069 SENIOR Explain the micro-batch model of Structured Streaming, and compare the default trigger, `processingTime`, `availableNow` and continuous processing.

- [ ] SP-070 SENIOR What is stored in a Structured Streaming checkpoint location (offsets, commits, state), and what happens if two queries share one checkpoint or someone deletes it?
- [ ] SP-071 SENIOR Compare the append, update and complete output modes, and say which of them work for a windowed aggregation with a watermark.
- [ ] SP-072 STAFF Which changes to a streaming query can you make while keeping its checkpoint, and which force a new one? Consider changing a filter, adding a stateful aggregation, and changing the number of shuffle partitions.
- [ ] SP-073 SENIOR How does `withWatermark` let Spark drop old state in a streaming aggregation, and what happens to an event that arrives after the watermark has passed its window?
- [ ] SP-074 STAFF A streaming job that uses `dropDuplicates` on `event_id` grows its state by 50 GB a day. How do you bound it, and what are the trade-offs of `dropDuplicatesWithinWatermark`?
- [ ] SP-075 STAFF Why would you move a large stateful streaming query from the default state store to the RocksDB state store provider, and which metrics tell you it is time?
- [ ] SP-076 SENIOR How do you use `foreachBatch` to MERGE each micro-batch into a table, and how do you make the write idempotent if Spark retries a batch?
- [ ] SP-077 SENIOR What is a stream-static join in Structured Streaming, and what does the stream see when the static dimension table is updated while the query is running?
- [ ] SP-078 STAFF Two Spark jobs write to the same Delta table at once and one fails with a concurrent modification error. Explain Delta's optimistic concurrency control, which operations conflict, and how you would design the writers to avoid it.
- [ ] SP-079 SENIOR Compare static and dynamic partition overwrite in Spark, and explain what Delta's `replaceWhere` adds when you rewrite one week of a table.
- [ ] SP-080 STAFF When writing to an Iceberg table from Spark, what does the write distribution mode (`none`, `hash`, `range`) control, and how does it change file counts and write speed?

Cluster sizing, cost, AWS and Kubernetes Week 9

- [ ] SP-081 STAFF How would you choose executor cores, executor memory and executor count for 20 nodes with 32 cores and 256 GB each, and why are about 5 cores per executor a common starting point?
- [ ] SP-082 STAFF A daily job costs 1,200 dollars per run. Walk through how you would cut that in half, from instance types and spot capacity to fixing the code itself.
- [ ] SP-083 STAFF What are the risks of running executors on spot instances, how does executor decommissioning reduce them, and should the driver ever run on spot?
- [ ] SP-084 SENIOR Why did the old rename-based output committers make Spark writes to S3 slow or unsafe, and what do the S3A magic committer and the EMRFS S3-optimized committer change?
- [ ] SP-085 STAFF Compare Amazon EMR on EC2, EMR on EKS, EMR Serverless, Databricks and self-managed Spark on Kubernetes for a team that runs 3,000 jobs a day.
- [ ] SP-086 SENIOR On Kubernetes, how are the Spark driver and executors created as pods, and how do pod templates, resource requests and limits, and node selectors affect scheduling?

- [ ] SP-087 STAFF How do you run dynamic allocation on Kubernetes without an external shuffle service, and what does shuffle tracking cost you in executors that cannot be released?
- [ ] SP-088 STAFF How would you measure and report Spark cost per team and per job on a shared platform, and which signals would you use to find waste, such as idle executors or low CPU use?

Debugging with the Spark UI Week 13

- [ ] SP-089 SENIOR Walk through the Jobs, Stages, Storage, Executors and SQL tabs of the Spark UI, and what you check first on each one when a job is slow.
- [ ] SP-090 SENIOR One stage has 2,000 tasks; the median task takes 20 seconds and the slowest takes 40 minutes. What does that tell you, and what do you look at next?
- [ ] SP-091 STAFF A job that ran in 40 minutes yesterday took 5 hours today with the same code. How do you find the cause using the Spark UI, the event log in the History Server, and input data stats?
- [ ] SP-092 SENIOR A job fails with `FetchFailedException` and lost executors during a large shuffle. What are the common root causes, and how do you fix each one?
- [ ] SP-093 STAFF The Spark UI shows tasks finishing quickly, but the application still takes 25 minutes, with long gaps between jobs where no tasks run. What could the driver be doing, such as listing files or planning, and how do you confirm it?
- [ ] SP-094 SENIOR How do you use the SQL tab's plan graph to find the operator that produced 10 times more rows than expected, for example after a bad join?

Staff-level Spark standards and migrations Weeks 14–15

- [ ] SP-095 STAFF You are asked to set Spark standards for an org of 50 data engineers. What would you standardize, such as configs, code patterns, testing, review checklists and cost limits, and how would you get teams to adopt them?
- [ ] SP-096 STAFF Plan the migration of 4,000 Hive SQL jobs from an on-premise Hadoop cluster to Spark on a cloud lakehouse. How do you sequence the work, validate results, and handle differences in SQL behavior?
- [ ] SP-097 STAFF How would you design a multi-tenant Spark platform where 30 teams share compute, so that one team's runaway job cannot starve everyone else?
- [ ] SP-098 STAFF How would you build automated performance regression checks for critical Spark jobs, so a code change that doubles shuffle size is caught before it reaches production?
- [ ] SP-099 STAFF When would you tell a team not to use Spark at all, and to use a warehouse SQL engine, DuckDB, or a plain Python service instead?
- [ ] SP-100 STAFF How would you evaluate moving 2,000 production jobs to Spark 4 or to a native vectorized engine such as Photon, Apache Gluten or Apache DataFusion Comet, and what would make you roll back?

TOPIC 6 OF 11

Data modeling, warehousing and lakehouse

100 questions · 52 Senior · 48 Staff

Dimensional modeling fundamentals Week 7

- [ ] DM-001 SENIOR What is the grain of a fact table, and why should you declare it before you choose any dimensions or measures?
- [ ] DM-002 STAFF You model e-commerce orders at the order-line grain. Where do order-level amounts like shipping cost and an order-wide discount go, and how do you allocate them without breaking totals?
- [ ] DM-003 SENIOR Compare a star schema and a snowflake schema, and explain why most teams on modern columnar warehouses lean toward star schemas.
- [ ] DM-004 SENIOR Explain transaction, periodic snapshot and accumulating snapshot fact tables, with one example of each from a ride-sharing business.
- [ ] DM-005 SENIOR What are additive, semi-additive and non-additive measures? Why can you not sum an account balance across days, and how should you store a ratio like conversion rate?
- [ ] DM-006 SENIOR What is a factless fact table, and how would you use one to find students who enrolled in a course but never attended a class?
- [ ] DM-007 SENIOR Explain degenerate, junk and role-playing dimensions, with a concrete example of each.
- [ ] DM-008 STAFF How do you model a many-to-many relationship, such as a bank account with several owners or a hospital visit with several diagnoses, using a bridge table, and how do you avoid double counting?
- [ ] DM-009 SENIOR What are conformed dimensions, and why do they matter when finance and marketing both report revenue by customer?
- [ ] DM-010 STAFF Build an enterprise bus matrix for five business processes at a subscription company. How does it guide which data marts you build first?
- [ ] DM-011 SENIOR Explain normalization up to third normal form, and why OLTP systems normalize while analytics models usually denormalize.
- [ ] DM-012 SENIOR How do you design a date dimension, which columns do you add for fiscal calendars and holidays, and why not just use date functions in every query?
- [ ] DM-013 STAFF A fact table must support both UTC and local-time reporting for users in 40 countries. How do you model time zones and daylight saving changes?
- [ ] DM-014 STAFF A team built one fact table per dashboard and now has 300 fact tables with overlapping logic. Explain why modeling business processes instead of reports avoids this, and how you would clean up the warehouse.

Slowly changing dimensions, keys and late data Week 7

- [ ] DM-015 SENIOR Explain SCD types 0, 1, 2 and 3, with an attribute in a customer dimension that fits each one.

- [ ] DM-016 SENIOR Design a Type 2 customer dimension: which columns do you add, such as a surrogate key, valid-from and valid-to dates and a current flag, and how does each fact row find the right version?
- [ ] DM-017 SENIOR Describe, step by step or in SQL, how a daily job applies Type 2 changes from a full extract of 50 million customers using a single MERGE statement.
- [ ] DM-018 STAFF Explain SCD types 4, 5 and 6, and describe a business question that a Type 6 hybrid answers easily but a plain Type 2 dimension does not.
- [ ] DM-019 STAFF Your Type 2 dimension creates a new version whenever any of 60 columns changes, and it now grows by 20 million rows a day. How do you decide which columns should create new versions, and what do you do with the rest?
- [ ] DM-020 SENIOR Compare surrogate keys and natural keys, and explain why a dimension keyed only on a source system's `customer_id` breaks when you add a second CRM.
- [ ] DM-021 STAFF In a distributed lakehouse, should surrogate keys be sequence integers or hashes of the natural key? Compare join speed, parallel loading, collision risk and reprocessing.
- [ ] DM-022 SENIOR What is a late-arriving fact, and how do you join it to the dimension version that was valid at event time rather than at load time?
- [ ] DM-023 SENIOR What is a late-arriving dimension, also called an early-arriving fact, and how does an inferred member row let you load the fact now and fill in the dimension later?
- [ ] DM-024 STAFF A source system backdates a customer's segment change by 3 months. How do you correct the Type 2 history and the facts already linked to the wrong version?
- [ ] DM-025 STAFF How would you build a Type 2 dimension from a CDC stream that can deliver several changes per key per day, sometimes out of order?
- [ ] DM-026 SENIOR How should a Type 2 dimension handle hard deletes in the source system, and how do you detect them if the source only sends full extracts?

Data Vault, one big table, events and the semantic layer Week 7

- [ ] DM-027 SENIOR Explain Data Vault's hubs, links and satellites, and what problem the approach is designed to solve.
- [ ] DM-028 STAFF When would you choose Data Vault over a Kimball star schema, and what does it cost you in query complexity and in the extra presentation layer you still need for BI?
- [ ] DM-029 STAFF Make the case for and against one big table for a product analytics team on a columnar warehouse, covering storage, update cost when a dimension changes, and metric consistency.
- [ ] DM-030 SENIOR How would you model raw product events with 300 event types whose properties change over time, and where do event-specific properties live?
- [ ] DM-031 SENIOR How do you build a sessions table from raw events using a 30-minute inactivity rule, and which window functions do you use?
- [ ] DM-032 STAFF Sessionization must handle events that arrive up to 2 days late and users on several devices. How does that change your design for a daily batch job versus a streaming job?
- [ ] DM-033 SENIOR What is a metrics layer or semantic layer, and what problem does it solve when three dashboards show three different numbers for "active users"?

- [ ] DM-034 STAFF Compare defining metrics in a semantic layer, such as the dbt Semantic Layer or LookML, with defining them as columns in gold tables. When would you use each?
- [ ] DM-035 SENIOR How do you model a funnel (visit → signup → first purchase) so analysts can compute conversion by weekly cohort and by acquisition channel?
- [ ] DM-036 STAFF How would you model the same event data for ML features and for BI reporting, including point-in-time correctness so training data does not leak future information?

Lakehouse table formats and architecture patterns Week 8

- [ ] DM-037 SENIOR Explain the medallion architecture: what level of cleaning and modeling belongs in bronze, silver and gold?
- [ ] DM-038 STAFF When does a strict bronze, silver and gold layout add cost without adding value, and how would you decide how many layers a domain really needs?
- [ ] DM-039 SENIOR Which problems do open table formats like Apache Iceberg, Delta Lake and Apache Hudi solve compared with plain Parquet files in folders?
- [ ] DM-040 SENIOR Explain Iceberg's metadata tree, from the table metadata file to manifest lists, manifest files and data files, and how a reader uses it to plan a query.
- [ ] DM-041 SENIOR How do snapshots enable time travel and isolated reads in a lakehouse table, and why does a long time travel window raise storage cost?
- [ ] DM-042 SENIOR Which schema changes are safe in Iceberg or Delta, such as add, drop, rename, reorder and widening a type, and why do column IDs or column mapping make renames safe?
- [ ] DM-043 STAFF Explain hidden partitioning and partition evolution in Iceberg. How would you move a 300 TB table from daily to hourly partitions without rewriting the old data?
- [ ] DM-044 STAFF Compare copy-on-write and merge-on-read for row-level updates and deletes, and pick one for a table that gets 5 million upserts an hour and is read by dashboards every minute.
- [ ] DM-045 SENIOR How does a `MERGE INTO` upsert work on a lakehouse table, and why can a MERGE that changes only a few thousand rows rewrite hundreds of files?
- [ ] DM-046 SENIOR Which maintenance jobs does a lakehouse table need, such as compaction, snapshot expiry, orphan file cleanup and Delta `VACUUM`, and what breaks if you skip them for 6 months?
- [ ] DM-047 STAFF Compare Iceberg, Delta Lake and Hudi for a company that uses Spark, Trino, Snowflake and Flink. Which factors matter most: engine support, catalog support, streaming upserts, or the community behind each?
- [ ] DM-048 STAFF What does a catalog do in a lakehouse, for example Hive Metastore, AWS Glue, an Iceberg REST catalog or Unity Catalog, and what goes wrong when two catalogs point at the same table?
- [ ] DM-049 SENIOR Compare the lambda and kappa architectures, and explain why open table formats with streaming upserts have made a single pipeline more practical.
- [ ] DM-050 STAFF A team wants to rebuild a 2 TB a day batch pipeline as a streaming pipeline. What do they gain and lose in correctness, cost, backfills and debugging, and how would you make the call?

Snowflake, BigQuery and Redshift Week 8

- [ ] DM-051 SENIOR Explain Snowflake's three layers: storage, virtual warehouses for compute, and cloud services. What does each one do?
- [ ] DM-052 SENIOR What are micro-partitions in Snowflake, what metadata does Snowflake keep for each one, and how does that metadata let queries prune data?
- [ ] DM-053 STAFF When should you define a clustering key on a Snowflake table, how do you measure clustering depth before and after, and what does automatic clustering cost?
- [ ] DM-054 SENIOR How do warehouse sizes, auto-suspend, auto-resume and multi-cluster warehouses work, and when do you scale up instead of scaling out?
- [ ] DM-055 SENIOR Compare Snowflake's result cache, the warehouse local disk cache, and queries answered from metadata alone, and explain why suspending a warehouse can make the next query slower.
- [ ] DM-056 SENIOR Compare bulk `COPY INTO`, Snowpipe and Snowpipe Streaming for loading data, including latency, how each is billed, and file size guidance.
- [ ] DM-057 STAFF Design an incremental pipeline in Snowflake with Streams and Tasks. What happens if the stream is not consumed before the source table's data retention period runs out?
- [ ] DM-058 STAFF When would you use Dynamic Tables instead of Streams and Tasks or dbt incremental models, and how does target lag affect cost and freshness?
- [ ] DM-059 STAFF How does zero-copy cloning work in Snowflake, what does a clone cost in storage as both copies change over time, and how would you use it to rehearse a risky schema migration on a 40 TB production database?
- [ ] DM-060 STAFF Snowflake spend grew from 80,000 to 200,000 dollars a month in two quarters. How would you find the drivers and bring it down, using account usage views, resource monitors and warehouse design?
- [ ] DM-061 SENIOR Compare Time Travel and Fail-safe in Snowflake, and how permanent, transient and temporary tables differ in retention and storage cost.
- [ ] DM-062 SENIOR Explain BigQuery's on-demand pricing by bytes scanned versus capacity pricing with slots, and how partitioning and clustering lower cost under each.
- [ ] DM-063 SENIOR Explain Redshift distribution styles (KEY, EVEN, ALL, AUTO) and sort keys, and what happens to a join when two large tables are distributed on different keys.
- [ ] DM-064 STAFF 300 analysts open dashboards at 9 a.m. every Monday. Compare how Snowflake multi-cluster warehouses, BigQuery reservations, and Redshift concurrency scaling with workload management handle that spike.

dbt and Airflow orchestration Week 8

- [ ] DM-065 SENIOR Explain dbt's core parts: models, `ref()` and `source()`, the view, table, incremental and ephemeral materializations, and how dbt builds its dependency graph.
- [ ] DM-066 SENIOR How does a dbt incremental model work, what does `is_incremental()` do, and how do you catch late-arriving rows with a lookback window?
- [ ] DM-067 STAFF Compare dbt incremental strategies such as `append`, `merge`, `delete+insert`, `insert_overwrite` and `microbatch`, and pick one for a 10 billion row event table on Snowflake and on BigQuery.

- [ ] DM-068 SENIOR What tests come built into dbt (`unique`, `not_null`, `accepted_values`, `relationships`), how are singular tests different, and what are dbt unit tests for?
- [ ] DM-069 SENIOR How does a dbt snapshot capture Type 2 history, and what is the difference between the `timestamp` and `check` strategies?
- [ ] DM-070 STAFF A dbt project has grown to 2,500 models and a full build takes 6 hours. How would you restructure it, and how would you change CI using state comparison and deferral to build only what changed?
- [ ] DM-071 STAFF How do dbt model contracts and model versions help when 15 teams depend on a shared `dim_customer` model and you need to change a column?
- [ ] DM-072 SENIOR What makes an Airflow task idempotent, and how do the logical date and the data interval help you write tasks that can rerun safely?
- [ ] DM-073 SENIOR Explain `catchup`, backfills and `max_active_runs` in Airflow, and what happens when you deploy a new daily DAG whose start date is 2 years ago.
- [ ] DM-074 STAFF You must backfill 18 months of a daily pipeline after a logic bug, without overloading the warehouse or breaking today's runs. How do you plan, run and validate it?
- [ ] DM-075 STAFF For a DAG that waits on 40 upstream tables, compare sensors in poke mode, sensors in reschedule mode, deferrable operators, and data-aware scheduling with Airflow datasets or assets.
- [ ] DM-076 STAFF Which Airflow anti-patterns would you block in code review, such as heavy top-level code, large XComs, or 500 copy-pasted DAGs, and what would you use instead?

Data quality, contracts, lineage and governance Week 8

- [ ] DM-077 SENIOR Name the main dimensions of data quality, such as freshness, volume, schema, uniqueness, validity and distribution, and give one test for each on an orders table.
- [ ] DM-078 STAFF How would you set up anomaly detection on row counts and key metrics for 3,000 tables without flooding the on-call channel with false alarms?
- [ ] DM-079 SENIOR What is a data contract between a producing service team and a data team, and what should it contain, such as schema, meaning of fields, SLAs, ownership and change process?
- [ ] DM-080 STAFF How would you roll out data contracts across 40 producer teams who do not report to you, and how would you enforce them in CI instead of by asking nicely?
- [ ] DM-081 SENIOR Explain the write-audit-publish pattern, and how branches or staging tables in a lakehouse let you check data before consumers can see it.
- [ ] DM-082 SENIOR What is data lineage, why is column-level lineage more useful than table-level lineage for impact analysis, and how does a standard like OpenLineage collect it?
- [ ] DM-083 STAFF What would make a data catalog actually used by 1,000 analysts, instead of becoming a stale list of tables nobody trusts?
- [ ] DM-084 SENIOR Compare role-based and attribute-based access control, and explain how row access policies and column masking policies protect PII in a warehouse.

- [ ] DM-085 STAFF A GDPR deletion request must remove one user from 300 tables, including lakehouse tables with time travel and backups. Design the end-to-end process and explain how you prove the data is gone.
- [ ] DM-086 STAFF Compare deleting rows, tokenizing PII, and crypto-shredding with per-user keys for handling 2 million right-to-be-forgotten requests a year.
- [ ] DM-087 SENIOR How would you find and tag PII columns across 20,000 tables, and keep the tags correct as teams add new columns?
- [ ] DM-088 STAFF How do you define and track SLAs or SLOs for data products, covering freshness, completeness and correctness, and what do you do when a tier-1 table misses its SLA three times in a month?

Debugging data correctness Week 13

- [ ] DM-089 SENIOR Daily revenue in the gold table is 3% higher than in the finance system. How do you trace the difference layer by layer back to the source?
- [ ] DM-090 SENIOR After a join to a Type 2 dimension, the fact table's row count doubled. What went wrong, and how do you write the join correctly?
- [ ] DM-091 STAFF A dbt incremental model has been silently dropping late rows for 2 months. How do you measure the gap, repair the history, and change the model so it cannot happen again?
- [ ] DM-092 SENIOR A Snowflake query that took 20 seconds now takes 15 minutes. How do you use the query profile to find the cause, such as poor pruning, spilling to remote storage, or an exploding join?
- [ ] DM-093 STAFF Query planning on an Iceberg table takes 4 minutes even though each query reads little data. What would you check in the table metadata, and how would you fix it?
- [ ] DM-094 STAFF A key dashboard metric dropped 40% overnight. How do you quickly tell whether it is a real business change, a pipeline failure, or an upstream definition change?

Staff-level architecture and platform decisions Weeks 14–15

- [ ] DM-095 STAFF Your company must choose between a Snowflake-centered warehouse and an open lakehouse on Databricks or Iceberg for 2 PB of data used by BI and ML training. How do you run the evaluation and make the decision?
- [ ] DM-096 STAFF Plan a migration from an on-premise Teradata warehouse with 8,000 tables and 600 reports to a cloud platform. How do you sequence domains, run old and new in parallel, and decide when to switch the old one off?
- [ ] DM-097 STAFF Compare data mesh with a central data platform team for a 2,000-person company. What would you keep central and what would you hand to domain teams?
- [ ] DM-098 STAFF Design a cost governance program for a data platform that spends 6 million dollars a year, covering tagging, showback or chargeback, budgets, and the incentives it creates for teams.
- [ ] DM-099 STAFF How would you design the data platform behind model training data at an AI company, covering dataset versioning, lineage from raw source to trained model, license and consent flags, and reproducible snapshots?
- [ ] DM-100 STAFF You have 12 months and 6 engineers to fix a warehouse that analysts no longer trust. What do you do in the first 90 days, and how do you show progress to leadership?

TOPIC 7 OF 11

AWS and Kubernetes

100 questions · 52 Senior · 48 Staff

S3 and storage Week 9

- [ ] CL-001 SENIOR Walk through the S3 storage classes from Standard to Glacier Deep Archive and say which data in a data lake you would put in each one.
- [ ] CL-002 SENIOR S3 has offered strong read-after-write consistency since late 2020. What did this change for Spark and Hive jobs that used to need S3Guard or EMRFS consistent view?
- [ ] CL-003 SENIOR What are the S3 request rate limits per prefix, and how does the way you design object keys affect throughput for a table with thousands of partitions?
- [ ] CL-004 STAFF A team plans to write millions of 10 KB objects to S3 every day. What does this do to request costs, listing time, lifecycle transition costs and query engines, and what would you tell them to change?
- [ ] CL-005 SENIOR Describe a lifecycle policy for a raw zone that moves objects to Standard-IA after 30 days, to Glacier Flexible Retrieval after 180 days, and deletes them after 2 years. Which minimum storage duration and minimum object size charges should you watch for?
- [ ] CL-006 SENIOR Why should every bucket that receives multipart uploads have a lifecycle rule that aborts incomplete multipart uploads?
- [ ] CL-007 STAFF When would you choose S3 Intelligent-Tiering over hand-written lifecycle rules, and when does its per-object monitoring fee make it a poor choice?
- [ ] CL-008 SENIOR How do S3 versioning and noncurrent version expiration work together, and how can versioning quietly double your storage bill?
- [ ] CL-009 STAFF Compare S3 Cross-Region Replication with having your pipeline write to two regions itself. Which failures does each one handle, and what does replication not copy by default?
- [ ] CL-010 SENIOR How would you use S3 Inventory and S3 Storage Lens to find which prefixes and teams drive your storage cost?
- [ ] CL-011 STAFF S3 now supports conditional writes. How could a table format or a pipeline use them to stop two writers from overwriting the same commit file?
- [ ] CL-012 STAFF When would you use S3 Express One Zone directory buckets for a data workload, and what durability and availability trade-off are you accepting?

IAM, encryption and governance Week 9

- [ ] CL-013 SENIOR Explain the difference between an IAM user, an IAM role and a role session. Why should pipelines almost never use long-lived access keys?
- [ ] CL-014 SENIOR How does IAM decide the result when identity policies, resource policies and SCPs contain a mix of explicit allows, explicit denies and no matching statement?
- [ ] CL-015 SENIOR What is a trust policy on an IAM role, and how is it different from the permissions policy attached to the same role?

- [ ] CL-016 STAFF A Spark job in account A must read a bucket in account B that is encrypted with a customer managed KMS key. List every policy that must allow the access and explain why each one is needed.
- [ ] CL-017 SENIOR Why do roles assumed by third-party vendors use an external ID, and what problem does it prevent?
- [ ] CL-018 STAFF How would you design least-privilege IAM for 200 Airflow DAGs without writing 200 hand-made policies?
- [ ] CL-019 STAFF What are permission boundaries, and when would a platform team use them to let product teams create their own IAM roles safely?
- [ ] CL-020 SENIOR How do IAM Roles for Service Accounts (IRSA) or EKS Pod Identity give a pod AWS credentials, and why is that better than relying on the node's instance profile?
- [ ] CL-021 SENIOR Compare SSE-S3, SSE-KMS and DSSE-KMS. What extra cost and throttling risk does SSE-KMS add, and how do S3 Bucket Keys help?
- [ ] CL-022 STAFF Explain envelope encryption with KMS. Why does KMS not encrypt your large data files directly?
- [ ] CL-023 SENIOR What does Lake Formation add on top of the Glue Data Catalog and IAM, and how do column-level and row-level filters work at a concept level?
- [ ] CL-024 STAFF Compare Lake Formation tag-based access control (LF-Tags) with granting permissions table by table. How does each approach scale to 5,000 tables and 50 teams?

Glue, Athena, EMR and Redshift Week 9

- [ ] CL-025 SENIOR What does the Glue Data Catalog store, and how do Athena, EMR, Redshift Spectrum and Glue jobs all use it as a shared Hive-compatible metastore?
- [ ] CL-026 STAFF When are Glue crawlers useful, and when do they cause trouble such as wrong schemas or too many tables? What would you use instead?
- [ ] CL-027 SENIOR How do Glue job bookmarks work, and what happens to incremental processing if someone reprocesses or deletes old files?
- [ ] CL-028 STAFF Compare Glue jobs, EMR on EC2, EMR on EKS and EMR Serverless for a nightly 5 TB Spark job. Which factors decide your choice?
- [ ] CL-029 SENIOR How is Athena priced, and what are four ways to cut the bytes a query scans?
- [ ] CL-030 SENIOR What is Athena partition projection, and why can it be faster than registering millions of partitions in the catalog?
- [ ] CL-031 STAFF How would you use Athena workgroups to set per-team query limits, cost controls and result locations for 30 analyst teams?
- [ ] CL-032 SENIOR Explain the roles of primary, core and task nodes in EMR on EC2. Why do teams usually run Spot capacity only on task nodes?
- [ ] CL-033 STAFF EMR Serverless offers pre-initialized capacity. When is it worth paying for warm capacity, and how do you set maximum capacity to protect the budget?
- [ ] CL-034 SENIOR Explain Redshift distribution styles (KEY, EVEN, ALL, AUTO) and sort keys. How do bad choices show up in query plans?

- [ ] CL-035 SENIOR What did RA3 nodes and Redshift managed storage change compared with older node types that tied storage to compute?
- [ ] CL-036 STAFF When would you pick Redshift Serverless over a provisioned RA3 cluster, and how do you compare RPU-hour pricing with steady, predictable usage?
- [ ] CL-037 STAFF Redshift, Athena on Iceberg, or Databricks SQL over the same S3 lake: how would you decide for a company with mixed BI and ad hoc workloads?
- [ ] CL-038 SENIOR What is Redshift Spectrum, and when does querying external tables on S3 beat loading the data into Redshift?

Streaming, serverless and orchestration Weeks 4, 9

- [ ] CL-039 SENIOR Explain Kinesis Data Streams shards, the per-shard write and read limits, and the difference between shared fan-out and enhanced fan-out.
- [ ] CL-040 STAFF Kinesis Data Streams or Amazon MSK for a 200 MB per second clickstream with 15 consumer teams? Compare cost, operations work, retention, replay and ecosystem.
- [ ] CL-041 SENIOR What does Amazon Data Firehose do, and when is it enough on its own instead of running your own consumer to land data in S3?
- [ ] CL-042 SENIOR Which Lambda limits matter for data work (maximum run time, memory, temporary disk, payload size, concurrency), and which kinds of data jobs do they rule out?
- [ ] CL-043 STAFF A team wants to process every new S3 object with a Lambda function. At 50,000 new objects per minute, what breaks first, and what would you design instead?
- [ ] CL-044 SENIOR Compare Step Functions Standard and Express workflows in maximum duration, execution guarantees and pricing model.
- [ ] CL-045 STAFF When would you orchestrate a pipeline with Step Functions instead of Airflow on MWAA, and when is the reverse true?
- [ ] CL-046 SENIOR How does MWAA load DAGs, plugins and Python requirements, and what usually goes wrong when someone changes the requirements file?
- [ ] CL-047 STAFF Would you run one shared MWAA environment for all teams, or one per team or domain? Compare cost, isolation, upgrade pain and blast radius.
- [ ] CL-048 STAFF How does Step Functions Distributed Map process millions of S3 objects in parallel, and when is a Spark job still the better tool?

Networking, observability and cost Week 9

- [ ] CL-049 SENIOR What is the difference between an S3 gateway endpoint and an S3 interface endpoint, both in cost and in how traffic is routed?
- [ ] CL-050 SENIOR Why do NAT gateways often become one of the largest line items for data platforms, and how do VPC endpoints reduce that cost?
- [ ] CL-051 STAFF Explain cross-AZ data transfer charges. How do they affect self-managed Kafka replication, Spark shuffle and multi-AZ databases, and when is it worth keeping a workload in one AZ?
- [ ] CL-052 SENIOR What is AWS PrivateLink, and how would you use it to let consumer accounts reach a shared data service without VPC peering?

- [ ] CL-053 SENIOR Which CloudWatch metrics and alarms would you set up for a Kinesis-to-S3 pipeline and for an EMR cluster?
- [ ] CL-054 STAFF CloudWatch Logs ingestion has become a large part of your bill. How would you reduce it without losing the ability to debug incidents?
- [ ] CL-055 SENIOR How are Spot Instances interrupted, how much warning do you get, and how do allocation strategies like price-capacity-optimized and spreading across many instance types reduce interruptions?
- [ ] CL-056 STAFF At a concept level, compare Compute Savings Plans, EC2 Instance Savings Plans and Reserved Instances. How would you decide how much to commit for a spiky data platform?
- [ ] CL-057 STAFF A partner wants to pull 500 TB a month from your S3 lake. How do egress charges work, and what options such as Requester Pays buckets or sharing within the same region would you consider?
- [ ] CL-058 STAFF How would you build cost visibility per team and per pipeline using cost allocation tags and the Cost and Usage Report, and what do you do about shared costs that cannot be tagged?
- [ ] CL-059 SENIOR What does a well-designed VPC for a data platform look like in terms of public and private subnets, AZ spread, endpoints and security groups?

Infrastructure as code Week 9

- [ ] CL-060 SENIOR What is Terraform state, why must it be stored remotely with locking, and what can happen if two engineers run apply at the same time without a lock?
- [ ] CL-061 SENIOR How do you handle drift when someone changes a Terraform-managed resource by hand in the AWS console?
- [ ] CL-062 STAFF How would you split Terraform modules and state files for a data platform with dev, staging and prod across 15 accounts, so that one bad apply cannot break everything?
- [ ] CL-063 STAFF How would you let product teams create their own buckets, Glue databases and IAM roles through infrastructure as code while the platform team keeps guardrails in place?
- [ ] CL-064 SENIOR How would you safely rename or move a Terraform resource that holds production data, such as an S3 bucket, without Terraform destroying it?

Kubernetes core Week 9

- [ ] CL-065 SENIOR Explain pods, Deployments, StatefulSets, DaemonSets and Jobs. Which would you use for a Kafka broker, a log shipper, a REST API and a nightly batch job?
- [ ] CL-066 SENIOR What do resource requests and limits do for CPU and for memory? Why does going over a memory limit kill the container while going over a CPU limit only slows it down?
- [ ] CL-067 SENIOR What are the Guaranteed, Burstable and BestEffort QoS classes, and which pods does the kubelet evict first when a node runs low on memory?
- [ ] CL-068 SENIOR What does `OOMKilled` with exit code 137 mean, and how do you tell a container hitting its own memory limit apart from a pod evicted because the node ran out of memory?
- [ ] CL-069 SENIOR Compare ClusterIP, NodePort, LoadBalancer and headless Services. Why do StatefulSets usually use a headless Service?

- [ ] CL-070 SENIOR How are ConfigMaps and Secrets different, and why is a Kubernetes Secret not secure by default? How would you handle database passwords properly?
- [ ] CL-071 SENIOR Explain liveness, readiness and startup probes. What goes wrong if a slow-starting JVM service has only an aggressive liveness probe?
- [ ] CL-072 SENIOR How does a rolling update work, what do `maxSurge` and `maxUnavailable` control, and how do readiness probes make it safe?
- [ ] CL-073 STAFF How does the Horizontal Pod Autoscaler decide the number of replicas, and why is CPU often the wrong signal for a Kafka consumer? What would you scale on instead?
- [ ] CL-074 STAFF How do the HPA and the cluster autoscaler or Karpenter work together, and why might you run low-priority placeholder pods to keep spare capacity warm?
- [ ] CL-075 SENIOR What are taints, tolerations, node selectors and node affinity, and how would you use them to keep batch jobs off nodes that run latency-sensitive services?
- [ ] CL-076 STAFF How would you set up namespaces, ResourceQuotas, LimitRanges, RBAC and NetworkPolicies so that 20 data teams can share one cluster safely?
- [ ] CL-077 SENIOR What is a PodDisruptionBudget, and how does it protect a StatefulSet during node upgrades or scale-down?
- [ ] CL-078 STAFF Shared clusters with namespace isolation versus one cluster per team: what are the security, cost and operations trade-offs?

Kubernetes for data and GPU workloads Weeks 9, 12

- [ ] CL-079 SENIOR How does Spark run on Kubernetes? Who creates the executor pods, how is the driver reached, and what takes over the role YARN used to play?
- [ ] CL-080 STAFF Spark on Kubernetes has no external shuffle service by default. How does that affect dynamic allocation, and what options do you have?
- [ ] CL-081 STAFF Compare running Spark on EMR on EC2 with running it on your own EKS cluster with a Spark operator. When does each one win?
- [ ] CL-082 SENIOR What is a Kubernetes operator, and how do custom resource definitions and a control loop let it manage a system like Kafka or Flink?
- [ ] CL-083 SENIOR How does a pod request a GPU in Kubernetes, what does the device plugin do, and why can a standard GPU request not be a fraction like 0.5?
- [ ] CL-084 STAFF Compare GPU sharing options such as NVIDIA MIG, time-slicing and one whole GPU per pod. Which would you pick for inference and which for training?
- [ ] CL-085 STAFF Why do distributed training jobs need gang scheduling, and what happens with the default scheduler when only half of the GPUs a job needs are free?
- [ ] CL-086 STAFF How would you design node pools for a cluster that runs Spark batch jobs, Airflow workers, Trino and GPU inference?
- [ ] CL-087 STAFF How would you use priority classes and preemption so production pipelines win over ad hoc notebooks when the cluster is full?
- [ ] CL-088 SENIOR How do StatefulSet pods with PersistentVolumeClaims behave when they are rescheduled to another node or another AZ, and why does this matter for EBS-backed volumes?

GCP and BigQuery comparisons Week 9

- [ ] CL-089 SENIOR Compare BigQuery on-demand pricing with capacity pricing based on slots and editions. When does each one make sense?
- [ ] CL-090 STAFF How do BigQuery partitioning and clustering compare with Redshift sort keys and with Iceberg partitioning on S3?
- [ ] CL-091 STAFF Compare the GCP resource hierarchy of organization, folders and projects with AWS Organizations and accounts for isolating data teams and their billing.
- [ ] CL-092 SENIOR Map these services across the two clouds and name one real difference for each pair: S3 and GCS, Kinesis and Pub/Sub, EMR and Dataproc, EKS and GKE.
- [ ] CL-093 STAFF A company runs its lake on AWS but its ML team wants to use BigQuery. What are the cost, latency, governance and egress concerns of this two-cloud setup?

Staff-level platform design Weeks 9, 14–15

- [ ] CL-094 STAFF Design a multi-account AWS strategy for a data platform. Which accounts would you create, what lives in each one, and which SCPs would you enforce across the organization?
- [ ] CL-095 STAFF Design a cost governance program for a data platform that spends 2 million USD a month. Cover budgets, tagging, showback or chargeback, anomaly alerts and who owns each action.
- [ ] CL-096 STAFF You are building a self-service data platform on AWS and Kubernetes for 20 product teams. What do you standardize as the golden path, and what do you leave to each team?
- [ ] CL-097 STAFF Set RPO and RTO targets for a data lake, a streaming pipeline and a warehouse. Which disaster recovery pattern (backup and restore, pilot light, warm standby or active-active) would you pick for each, and why?
- [ ] CL-098 STAFF A new law requires EU customer data to stay in the EU. How would you enforce data residency across S3, KMS keys, logs, backups, query engines and support access?
- [ ] CL-099 STAFF How would you plan the migration of 3 PB of data and 400 pipelines from an on-prem Hadoop cluster to AWS without a long freeze on new work?
- [ ] CL-100 STAFF Your platform team must choose between building on managed AWS services and running open-source engines on Kubernetes. Which criteria would you put in the design doc?

TOPIC 8 OF 11

ML, AI engineering and AI infrastructure

100 questions · 52 Senior · 48 Staff

ML fundamentals Week 10

- [ ] AI-001 SENIOR Explain supervised, unsupervised and self-supervised learning, with one data engineering example of each.
- [ ] AI-002 SENIOR What is overfitting, how do you spot it from training and validation curves, and what are three ways to reduce it?
- [ ] AI-003 SENIOR Why do we split data into train, validation and test sets, and what goes wrong if you tune hyperparameters on the test set?
- [ ] AI-004 SENIOR Why is a random split wrong for time-series or event data, and how would you build a time-based split instead?
- [ ] AI-005 SENIOR Give three concrete examples of label leakage in a churn model, and explain how a data engineer can prevent each one in the pipeline.
- [ ] AI-006 SENIOR Define precision and recall. For a fraud model, which one would the business care about more, and why?
- [ ] AI-007 SENIOR What does ROC AUC measure, and why can PR AUC be more useful when positives are only 0.1 percent of the data?
- [ ] AI-008 SENIOR When would you report RMSE instead of MAE for a regression model, and how do outliers affect each one?
- [ ] AI-009 STAFF A fraud dataset has 1 positive per 1,000 rows. Compare downsampling, upsampling, class weights and threshold tuning, and explain what each does to the predicted probabilities.
- [ ] AI-010 SENIOR What makes a good feature? Give examples of aggregations, ratios, time-since features and encodings for high-cardinality categories.
- [ ] AI-011 STAFF How would you choose a decision threshold for a classifier when false positives and false negatives have very different business costs?
- [ ] AI-012 SENIOR What is the difference between data drift, concept drift and label drift? Give a real-world example of each.
- [ ] AI-013 SENIOR Explain the bias-variance trade-off and how model complexity and the amount of training data affect each side.
- [ ] AI-014 STAFF What is model calibration, and why does it matter when a model's score is used as a probability by a downstream system?

Feature stores and training-serving skew Week 10

- [ ] AI-015 SENIOR What problem does a feature store solve, and what is the difference between its offline store and its online store?
- [ ] AI-016 SENIOR What is point-in-time correctness, and how does an "as of" join stop future data from leaking into training rows?

- [ ] AI-017 STAFF Design a point-in-time join of 500 million labeled events against 200 feature tables in Spark. How do you keep it both correct and affordable?
- [ ] AI-018 SENIOR What is training-serving skew? Give three ways it can happen even when both sides use the same feature names.
- [ ] AI-019 STAFF Would you share one feature code path between batch training and online serving, or log online features and train on those logs? Discuss the trade-offs.
- [ ] AI-020 SENIOR How would you load batch features into an online store such as Redis or DynamoDB, and how do you handle TTLs and stale values?
- [ ] AI-021 STAFF How would you support a streaming feature such as "number of transactions in the last 5 minutes" with low serving latency and correct backfills?
- [ ] AI-022 STAFF Should your company build its own feature store or adopt one, such as Feast or a managed offering from a cloud or lakehouse vendor? What would you need to see to decide?
- [ ] AI-023 STAFF How do you version features and change a feature's definition without breaking models that are already in production?
- [ ] AI-024 STAFF What metadata should a feature registry hold so that a new team can find, trust and reuse a feature built by someone else?

MLOps and model monitoring Week 10

- [ ] AI-025 SENIOR What should experiment tracking record for each training run so that someone can reproduce the result six months later?
- [ ] AI-026 SENIOR What is a model registry, and how do stages or aliases such as staging and production help control deployments?
- [ ] AI-027 STAFF Design a CI/CD pipeline for models. Which tests run on the data, on the training code and on the trained model before it can be promoted?
- [ ] AI-028 SENIOR Compare batch scoring, online real-time inference and streaming inference, and give one use case for each.
- [ ] AI-029 STAFF How would you roll out a new model safely using shadow mode, canary traffic and A/B tests, and what would make you roll back?
- [ ] AI-030 SENIOR What would you monitor for a model in production when the true labels only arrive 30 days later?
- [ ] AI-031 STAFF How do PSI or KL divergence work as drift metrics, and how do you avoid alert fatigue when you monitor hundreds of features?
- [ ] AI-032 STAFF How would you design a feedback loop that captures predictions, features and later outcomes so that models can be retrained and audited?
- [ ] AI-033 STAFF When should retraining run on a fixed schedule and when should drift or a drop in performance trigger it? What are the risks of fully automatic retraining?
- [ ] AI-034 STAFF Why should you log the exact model version and feature values used for each prediction, and what does that cost at 50,000 predictions per second?

LLM fundamentals Week 11

- [ ] AI-035 SENIOR What is a token, and why does the same text produce different token counts across models and languages?
- [ ] AI-036 SENIOR What is a context window, and what are the cost and quality trade-offs of putting a very long document into the prompt?
- [ ] AI-037 SENIOR What do temperature and top-p change during sampling, and what settings would you use for data extraction versus creative writing?
- [ ] AI-038 SENIOR What is an embedding, and why can the cosine similarity between two embeddings stand in for "these texts mean similar things"?
- [ ] AI-039 STAFF Explain the difference between pretraining, fine-tuning and prompting. When would a data team choose fine-tuning over better prompts or RAG?
- [ ] AI-040 STAFF How would you get reliable structured JSON output from an LLM in a pipeline that extracts fields from 10 million documents?
- [ ] AI-041 SENIOR Why do LLMs hallucinate, and which parts of a system design reduce hallucinations?
- [ ] AI-042 STAFF How would you estimate the monthly cost of an LLM feature from requests per day, input tokens, output tokens and cache hit rate? Which number usually dominates?

RAG and vector search Week 11

- [ ] AI-043 SENIOR Walk through the main steps of a RAG pipeline, from document ingestion to the final answer, and say what can fail at each step.
- [ ] AI-044 SENIOR Compare fixed-size, recursive, sentence-based and structure-aware chunking. How do chunk size and overlap affect retrieval quality?
- [ ] AI-045 STAFF How would you choose an embedding model for an internal search product? What would you test besides a public leaderboard score?
- [ ] AI-046 SENIOR What is approximate nearest neighbor search, and why don't we run exact search over 100 million vectors?
- [ ] AI-047 SENIOR Explain at a high level how an HNSW index works and what the `M`, `efConstruction` and `efSearch` parameters trade off.
- [ ] AI-048 STAFF Compare HNSW, IVF and product quantization indexes in memory use, build time, recall and the cost of updates.
- [ ] AI-049 SENIOR What is hybrid search, and how would you combine BM25 keyword scores with vector scores, for example with reciprocal rank fusion?
- [ ] AI-050 SENIOR What does a reranker such as a cross-encoder add after the first retrieval step, and what does it cost in latency?
- [ ] AI-051 STAFF How would you choose between a dedicated vector database, pgvector in Postgres, a search engine like OpenSearch, and vector search built into your lakehouse?
- [ ] AI-052 STAFF Source documents change every hour. How would you keep a vector index fresh, handle deletes, and avoid serving answers from removed documents?
- [ ] AI-053 STAFF How do you enforce document-level access control in RAG so a user never sees chunks they are not allowed to read? Compare filtering before retrieval with filtering after it.

- [ ] AI-054 SENIOR What metadata would you store with each chunk, and how do metadata filters interact with recall in ANN search?
- [ ] AI-055 STAFF How would you evaluate a RAG system separately for retrieval quality and for answer quality? Name the metrics for each.
- [ ] AI-056 SENIOR How would you build a golden evaluation set for a RAG assistant over company documents, and how many examples would you start with?
- [ ] AI-057 STAFF You must move 2 billion chunks to a new embedding model. How do you plan the re-embedding work, the cost and the cutover?
- [ ] AI-058 STAFF What are the trade-offs between storing raw chunk text in the vector database and storing only IDs that point to text in another store?

Agents, tool use and evals Week 11

- [ ] AI-059 SENIOR What is an LLM agent, and how is tool use (function calling) different from a single prompt and response?
- [ ] AI-060 SENIOR What is the Model Context Protocol (MCP) at a concept level, and what problem does it solve when connecting models to tools and data?
- [ ] AI-061 STAFF You are asked to give an agent access to the company warehouse through a SQL tool. What guardrails would you add for safety, cost and correctness?
- [ ] AI-062 SENIOR What is prompt injection, and how can text inside a retrieved document or a tool result try to take control of an agent?
- [ ] AI-063 STAFF Which guardrails would you put on inputs, on tool calls and on outputs for an agent that is allowed to write to production systems?
- [ ] AI-064 SENIOR How do you write tool names, descriptions and input schemas so that a model picks the right tool and passes valid arguments?
- [ ] AI-065 SENIOR What is an offline eval set for an LLM feature, and how is it different from a unit test?
- [ ] AI-066 STAFF What are the known pitfalls of LLM-as-judge, such as position bias, length bias and self-preference, and how do you check that a judge agrees with human raters?
- [ ] AI-067 STAFF How would you build an eval pipeline that runs on every prompt, model or retrieval change, and what should it store so results can be compared over time?
- [ ] AI-068 SENIOR What should you log for every agent run so that you can debug a bad outcome later?
- [ ] AI-069 STAFF When is a multi-step agent the wrong design, and a fixed workflow with a few LLM calls the better choice?
- [ ] AI-070 SENIOR How do you measure the cost and latency of an agent task that makes a different number of model and tool calls each time?

LLM serving and AI infrastructure Week 12

- [ ] AI-071 SENIOR What is the KV cache, and why does GPU memory use grow with both batch size and sequence length?
- [ ] AI-072 SENIOR Estimate the GPU memory needed to serve a 70 billion parameter model with 16-bit weights. What else besides the weights needs memory?

- [ ] AI-073 SENIOR Explain static batching versus continuous batching for LLM inference, and why continuous batching raises throughput.
- [ ] AI-074 STAFF Explain the trade-off between latency and throughput in LLM serving. How do time to first token and tokens per second each change as batches get larger?
- [ ] AI-075 SENIOR What is quantization, for example to 8-bit or 4-bit weights, and what do you gain and risk compared with 16-bit weights?
- [ ] AI-076 STAFF When would you use tensor parallelism, pipeline parallelism or simply more replicas to serve a large model?
- [ ] AI-077 SENIOR What are the prefill and decode phases of LLM inference, and why do they stress the GPU in different ways?
- [ ] AI-078 STAFF How would you autoscale LLM inference servers when a cold start means loading 100 GB of weights? Which signal would you scale on?
- [ ] AI-079 STAFF How would you calculate the cost per 1,000 requests for a self-hosted model, and when does self-hosting beat a hosted API?
- [ ] AI-080 SENIOR Compare exact-match response caching, prompt prefix caching and semantic caching. What are the correctness risks of semantic caching?
- [ ] AI-081 SENIOR What is PagedAttention, as used in vLLM, and which GPU memory problem does it solve?
- [ ] AI-082 STAFF Design a batch inference pipeline that runs an LLM over 500 million rows in the lake. How do you handle retries, rate limits, partial failures and cost?
- [ ] AI-083 SENIOR What does speculative decoding do, and when does it reduce latency?
- [ ] AI-084 STAFF How would you plan GPU capacity so that a fixed pool is shared between training, batch inference and online serving across several teams?

Training data pipelines Week 12

- [ ] AI-085 SENIOR Walk through a pipeline that turns raw web crawl data into a training-ready text dataset. What are the main stages?
- [ ] AI-086 SENIOR Explain how MinHash with locality-sensitive hashing finds near-duplicate documents, and why exact hashing is not enough.
- [ ] AI-087 STAFF How would you run near-duplicate detection over 10 billion documents in Spark? Where are the shuffle and memory bottlenecks?
- [ ] AI-088 STAFF Which quality filters would you apply to web text, such as language ID, heuristic rules, classifier scores and perplexity, and how could each one bias the data?
- [ ] AI-089 SENIOR How would you detect and remove PII such as emails, phone numbers and government IDs from a training corpus, and how do you measure the recall of the PII filter?
- [ ] AI-090 SENIOR Where does tokenization fit in the data pipeline, and why should tokenized shards be versioned together with the tokenizer?
- [ ] AI-091 STAFF How would you make a training dataset fully reproducible, so that a run from three months ago can be rebuilt exactly? Which lineage would you record?
- [ ] AI-092 STAFF What is a data mixture, and how would you build tooling that lets researchers change mixture weights without re-running the whole pipeline?

- [ ] AI-093 STAFF How do you keep eval benchmark data out of the training data (decontamination), and how would you show that it worked?
- [ ] AI-094 SENIOR Why do training data loaders prefer large sharded files in formats such as Parquet or WebDataset, and what goes wrong with millions of small files?

Staff-level AI platform strategy Weeks 14–15

- [ ] AI-095 STAFF Your company wants an internal RAG assistant. Would you build it from open-source parts or buy a vendor product? Write down your decision criteria.
- [ ] AI-096 STAFF How should a data platform team support AI teams? Which parts do you offer as shared services, and which do you leave to each AI team?
- [ ] AI-097 STAFF How would you handle a user's data deletion request when their data may sit in training sets, embeddings, caches and fine-tuned models?
- [ ] AI-098 STAFF Design a governance process that decides which datasets can be used for training, covering licenses, consent, retention and approvals.
- [ ] AI-099 STAFF Should prompts and LLM responses be logged for quality work and debugging? How do you balance that with privacy and retention rules?
- [ ] AI-100 STAFF How would you decide whether to send company data to a third-party model API, use a private deployment, or self-host an open-weight model?

TOPIC 9 OF 11

Debugging and production scenarios

100 questions · 52 Senior · 48 Staff

Spark performance and failures Weeks 5–6, 13

- [ ] DB-001 SENIOR Your daily Spark job went from 1 hour to 3 hours overnight with no code change. Walk me through how you find the cause.
- [ ] DB-002 SENIOR In the Spark UI, one stage has 199 tasks that finish in 30 seconds and 1 task that runs for 40 minutes. What do you check, and how do you fix it?
- [ ] DB-003 STAFF A join between a 2 TB events table and a 50 GB users table is skewed because 5 percent of events have a null or "unknown" user ID. Walk through three fixes and when you would pick each.
- [ ] DB-004 SENIOR Executors keep failing with "Container killed by YARN for exceeding memory limits". What is actually running out of memory, and which settings and code changes do you try first?
- [ ] DB-005 SENIOR The driver crashes with an OutOfMemoryError near the end of a job. Which code patterns commonly cause driver OOM, and how do you confirm which one it is?
- [ ] DB-006 SENIOR The Spark UI shows very large "spill (disk)" numbers on a shuffle stage. What does that tell you, and what would you change?
- [ ] DB-007 STAFF A table written by your job now has 400,000 files averaging 200 KB each, and every downstream reader is slow. How did this happen, and how do you fix both the writer and the existing data?
- [ ] DB-008 SENIOR A query that used to read one day of data now scans the whole table. How do you confirm that partition pruning stopped working, and what are the usual causes?
- [ ] DB-009 SENIOR A few tasks in every stage run 5 times slower than the rest, on different keys each time. How do you tell a bad node from data skew, and when does speculative execution help?
- [ ] DB-010 STAFF After a Spark version upgrade, a job produces slightly different row counts. How do you find which behavior change caused it and prove which output is correct?
- [ ] DB-011 SENIOR Adaptive query execution is turned on, but a skewed join is still slow. What would you check in the physical plan and in the AQE skew join settings?
- [ ] DB-012 STAFF A broadcast join that worked for a year now fails with a timeout or a driver OOM. What changed, and how do you make the job resilient to this kind of data growth?
- [ ] DB-013 SENIOR A job fails with a FetchFailedException during a shuffle after running for 2 hours. What are the likely causes, and how do you narrow them down?
- [ ] DB-014 STAFF Your Spark compute costs doubled this quarter, but data volume grew only 20 percent. How do you find where the money went and which jobs to fix first?

Kafka incidents Weeks 4, 13

- [ ] DB-015 SENIOR Consumer lag on a critical topic grew from near zero to 2 hours over the morning. Walk me through how you decide whether the cause is the producer, the brokers or the consumer.

- [ ] DB-016 STAFF Downstream teams report duplicate records in a table fed by a Kafka consumer. List the ways duplicates can enter the pipeline and how you would find which one is happening.
- [ ] DB-017 STAFF Your consumer group rebalances every few minutes and throughput has dropped to almost nothing. What causes rebalance storms, and which settings and client changes stop them?
- [ ] DB-018 SENIOR One partition of a topic gets 10 times the traffic of the others, and its consumer is always behind. How do you confirm a hot key, and what are your options?
- [ ] DB-019 STAFF After a broker failure, a team says some messages they got an acknowledgment for are gone. Which producer and topic settings make this possible, and how do you prove what happened?
- [ ] DB-020 STAFF Events for the same user arrive out of order in your consumer. What can cause this in a Kafka pipeline, and how do you design the consumer to cope with it?
- [ ] DB-021 SENIOR Producers start getting timeout errors and request latency is high, but broker CPU looks normal. What else do you check?
- [ ] DB-022 STAFF A consumer committed offsets before writing its results and then crashed. How do you work out which records were lost, recover them, and stop this from happening again?
- [ ] DB-023 SENIOR A new consumer group was deployed with `auto.offset.reset` set to `latest`, and the team says data is missing. Explain what happened and how you would backfill.
- [ ] DB-024 STAFF Disk usage on the brokers is growing fast and one broker is almost full. How do you find the cause, buy time safely, and fix the root problem?
- [ ] DB-025 SENIOR A producer schema change caused every consumer to fail deserialization. How do you restore service quickly, and what process stops this next time?

Streaming state and late data Weeks 4, 13

- [ ] DB-026 SENIOR Your streaming job's hourly counts are always about 3 percent lower than a batch recompute of the same hour. How do you find out whether late events are being dropped?
- [ ] DB-027 STAFF Someone set the watermark to 10 minutes, but mobile events often arrive 2 hours late. What symptoms would you see, and how would you pick a better watermark and handle the very late data?
- [ ] DB-028 STAFF A Structured Streaming job will not restart after a deploy and reports a checkpoint error. What could have corrupted or invalidated the checkpoint, and how do you recover without losing or duplicating data?
- [ ] DB-029 STAFF The state store of a streaming join grows every day until executors run out of memory. Which design mistakes cause unbounded state, and how do you fix them?
- [ ] DB-030 SENIOR A Flink job's checkpoints started taking 10 minutes and are now timing out. What do you look at first?
- [ ] DB-031 SENIOR After a code change, a streaming job restarts from its checkpoint but now produces wrong aggregates. Which kinds of changes are not safe to make with existing state?
- [ ] DB-032 STAFF A streaming job commits to a Delta or Iceberg table every 30 seconds, and readers of that table are getting slower each day. What is happening, and how do you fix it without adding much latency?

- [ ] DB-033 SENIOR A streaming job's processing time per micro-batch keeps growing even though the input rate is flat. What are the possible causes?
- [ ] DB-034 STAFF A stream-stream join between orders and payments misses about 1 percent of matches. How do you find out whether the cause is the watermarks, the join keys or event timing?

Batch and orchestration Week 13

- [ ] DB-035 SENIOR An Airflow task has shown as "running" for 9 hours, but the worker has no process for it. What are the possible causes, and how do you clean it up safely?
- [ ] DB-036 SENIOR Many Airflow tasks sit in "queued" and never start, although the workers look idle. What do you check?
- [ ] DB-037 SENIOR A 30-day backfill finished, and now the target table has twice the expected rows for those days. How did that happen, and how do you repair the data?
- [ ] DB-038 STAFF Your team's pipelines are not idempotent, and every retry risks duplicates. How would you change the write patterns across 150 jobs, and in what order?
- [ ] DB-039 SENIOR Every year when daylight saving time changes, one daily job either skips a day or runs twice. How do you find the bug and fix it for good?
- [ ] DB-040 SENIOR A daily report shows a drop for "yesterday" every morning that fixes itself by noon. How do you tell whether this is a timezone bug, late-arriving data or a scheduling problem?
- [ ] DB-041 STAFF A job loaded only 60 percent of a day's files because the upstream drop was still in progress. How do you detect partial loads, and how do you redesign the pipeline to wait for complete data?
- [ ] DB-042 STAFF An upstream team re-sent a corrected file for last Tuesday. Walk through how you find and rerun every downstream job and table that is affected.
- [ ] DB-043 SENIOR A task fails at random about once a week with a connection reset from the source database. How do you decide between adding retries, changing timeouts and fixing the source?
- [ ] DB-044 STAFF The Airflow scheduler is slow, DAG parsing takes minutes and tasks start late. What causes this, and what would you change in the DAG code and the deployment?
- [ ] DB-045 STAFF Two pipelines write to the same table and sometimes overwrite each other's partitions. How do you prove it is happening, and how do you redesign ownership?

Data quality and correctness Weeks 7, 13

- [ ] DB-046 SENIOR The row count of a fact table fell 8 percent from yesterday, but no job failed. How do you find where rows are being silently dropped?
- [ ] DB-047 SENIOR An inner join quietly drops rows because the dimension table is missing new keys. How would you catch this kind of silent row loss in the future?
- [ ] DB-048 STAFF An upstream service added a column and renamed another, and your pipeline still reports success. What breaks downstream, and how do you detect schema drift early?
- [ ] DB-049 SENIOR The null rate of a key column jumped from 0.1 percent to 30 percent overnight. How do you trace where the nulls came from?

- [ ] DB-050 SENIOR Revenue in a report is 4 times higher than expected after a new join was added. How do you prove that duplicate keys are exploding the join, and how do you fix it?
- [ ] DB-051 STAFF The Finance and Product dashboards show different numbers for "active users" last month. How do you find the reason, and how do you get the company to one number?
- [ ] DB-052 STAFF A data quality check fails every night, and the team has learned to ignore it. How do you fix both the checks and the habits around them?
- [ ] DB-053 STAFF A currency conversion bug inflated revenue for 3 weeks before anyone noticed. How do you find the affected rows and correct the historical tables and reports?
- [ ] DB-054 STAFF After a bad incident, leadership asks you to add data quality coverage to 2,000 tables. How do you catch real problems without flooding teams with false alerts?
- [ ] DB-055 SENIOR A dedup step uses `row_number()` over a key ordered by timestamp, and on some days it keeps the wrong record. What is going wrong?
- [ ] DB-056 SENIOR A CDC pipeline from Postgres still shows rows in the lake that were deleted in the source. How do you find where the deletes got lost?
- [ ] DB-057 STAFF An ML team finds that a feature table has been off by one day for six months. How do you size the impact, fix it, and tell the affected teams?

Warehouse, lakehouse and SQL Weeks 7–8, 13

- [ ] DB-058 SENIOR A dashboard query that ran in 5 seconds now takes 2 minutes. Walk through how you use the query plan and the query profile to find the cause.
- [ ] DB-059 SENIOR Your warehouse bill jumped 60 percent in one week. How do you find which users, queries or jobs caused it?
- [ ] DB-060 STAFF Queries on a clustered table get slower every week as new data lands. Explain clustering decay, and how you would fix it and keep it fixed.
- [ ] DB-061 SENIOR An analyst's query uses `SELECT DISTINCT` on top of a large join, runs very slowly and returns the wrong totals. What do you look for?
- [ ] DB-062 SENIOR A query on an Iceberg table is slow because planning takes longer than execution. What do you check in the table metadata, and which maintenance fixes it?
- [ ] DB-063 STAFF A daily MERGE into a Delta table gets slower every day and now takes 3 hours. How do you find the reason and speed it up?
- [ ] DB-064 STAFF Two concurrent writers to a Delta or Iceberg table keep failing with commit conflicts. What causes this, and how would you change the write pattern?
- [ ] DB-065 SENIOR Someone ran a DELETE without a WHERE clause on a production table. Walk through your recovery options in a lakehouse table and in a cloud warehouse.
- [ ] DB-066 STAFF Ad hoc queries from one team keep slowing down scheduled ELT jobs in a shared warehouse. How do you prove there is contention, and how do you isolate the workloads?
- [ ] DB-067 SENIOR Old snapshots were expired on a table, and now a downstream incremental job fails. What happened, and how do you recover?

Cloud and Kubernetes Weeks 9, 13

- [ ] DB-068 SENIOR Spark executor pods on Kubernetes keep getting `OOMKilled` even though the JVM heap is well under its maximum. What is using the memory, and how do you fix the pod sizing?
- [ ] DB-069 SENIOR A Python service in a pod has high latency, but dashboards show CPU usage well under its limit. How would you confirm CPU throttling and fix it?
- [ ] DB-070 SENIOR Your Spark job writing to S3 starts failing with 503 Slow Down errors. Walk through the cause and the fixes on both the writer side and the key layout.
- [ ] DB-071 SENIOR A Glue job that worked yesterday now fails with Access Denied on an S3 read. Walk through how you find which policy is denying the request.
- [ ] DB-072 STAFF The NAT gateway bill tripled this month. How do you find which workloads are sending traffic through it, and what do you change?
- [ ] DB-073 SENIOR Pods for a new deployment stay in Pending for 20 minutes. What are the possible reasons, and which commands and events do you check?
- [ ] DB-074 SENIOR A deployment goes into CrashLoopBackOff right after a config change. How do you find the cause and roll back safely?
- [ ] DB-075 SENIOR At peak hours your pipelines start failing with throttling errors from AWS APIs such as the Glue Data Catalog. How do you find which calls are throttled and reduce them?
- [ ] DB-076 STAFF A long EMR job on Spot capacity keeps failing in its last hour when Spot nodes are reclaimed. How do you make it finish reliably without moving everything to On-Demand?
- [ ] DB-077 SENIOR After every rolling deploy, about 2 percent of requests to a service fail with connection errors for a minute. How do you find the cause?
- [ ] DB-078 STAFF A Terraform plan for production wants to destroy and recreate a database. How do you investigate before anyone approves the apply?

AI systems Weeks 11–12, 13

- [ ] DB-079 STAFF After the weekly re-index, users say the RAG assistant's answers got worse. How do you find out what changed in the data, the chunking or the index?
- [ ] DB-080 SENIOR Right after a deploy, vector search starts returning nearly random results, but no errors are logged. What would you check first, and how would you stop this from happening again?
- [ ] DB-081 STAFF P99 latency of your LLM endpoint jumped from 2 seconds to 12 seconds while average latency barely moved. How do you investigate?
- [ ] DB-082 SENIOR An inference server crashes with CUDA out of memory, but only on some requests. Which request properties do you look at, and which server settings would you change?
- [ ] DB-083 STAFF A fine-tuning job fails with GPU out of memory at step 4,000 but not at step 1. What could make memory grow during training, and how do you find it?
- [ ] DB-084 STAFF The eval score of your LLM feature dropped 6 points after a prompt change that "should not matter". How do you tell a real regression from eval noise, and what do you do next?
- [ ] DB-085 SENIOR An agent sometimes loops, calling the same tool 40 times until it hits the step limit. How do you debug it and stop it from happening?

- [ ] DB-086 STAFF A RAG assistant showed a user a snippet from a document they had no access to. How do you contain the incident and find the gap in the access control path?
- [ ] DB-087 SENIOR Your LLM API bill doubled in a week while user traffic stayed flat. How do you find the cause?
- [ ] DB-088 STAFF A batch LLM job over 50 million rows is 80 percent done and keeps failing on rate limits and timeouts. How do you finish it without reprocessing rows or paying twice?
- [ ] DB-089 STAFF The answer quality of a hosted model seems to have changed even though nothing changed on your side. How do you confirm it, and how do you protect your product from this in the future?
- [ ] DB-090 SENIOR A fraud model's precision fell by half in production right after its feature pipeline moved to a new engine. How do you prove or rule out training-serving skew?

Incident leadership Weeks 13–15

- [ ] DB-091 STAFF You are the incident commander for a data outage that broke the executive revenue dashboard on the morning of an earnings call. Walk through your first 30 minutes.
- [ ] DB-092 SENIOR How do you decide the severity of a data incident? Give an example of a SEV1 and a SEV3 for a data platform.
- [ ] DB-093 STAFF Write the first status update you would send to stakeholders when you know the data is wrong but not yet why. What do you include, and what do you leave out?
- [ ] DB-094 STAFF During an incident, when do you stop looking for the root cause and instead roll back or switch to a known-good snapshot?
- [ ] DB-095 STAFF Walk through a blameless postmortem for a pipeline that published wrong numbers for 3 days. What makes the action items actually get done?
- [ ] DB-096 STAFF The same type of incident has happened three times this quarter in different teams. How do you drive a fix across teams you do not manage?
- [ ] DB-097 SENIOR After a fix, how do you verify that the data is correct again, and how do you decide when to tell users they can trust the dashboard?
- [ ] DB-098 STAFF After a string of incidents, you are asked to set data SLOs for freshness, completeness and correctness on critical tables. How do you pick them, and how do they change on-call and alerting?
- [ ] DB-099 SENIOR You are paged at 3 a.m. for a failed job that feeds a report used at 9 a.m. How do you triage, and how do you decide whether to fix it now or wait?
- [ ] DB-100 STAFF Tell me about the worst data incident you led: what happened, what you did in the first hour, and what you changed afterward so it could not happen again.

TOPIC 10 OF 11

System design

100 questions · 35 Senior · 65 Staff

Ingestion and event pipelines Weeks 3–4

- [ ] SD-001 SENIOR Design clickstream ingestion for 1 million events a second, with events arriving up to 3 days late from offline mobile clients.
- [ ] SD-002 SENIOR Design a nightly batch load of 2,000 tables from 30 OLTP databases into a warehouse, where the load must finish by 6 a.m. and some tables have no last-updated column.
- [ ] SD-003 STAFF Design change data capture from 500 sharded MySQL databases into a lakehouse, keeping tables less than 5 minutes behind the source while schema changes happen without warning.
- [ ] SD-004 SENIOR Design CDC from a single 20 TB Postgres database into Kafka and then Iceberg, where the initial snapshot must not slow down production queries.
- [ ] SD-005 SENIOR Design hourly ingestion from 200 third-party SaaS APIs such as Salesforce, Stripe and Zendesk, each with different rate limits, pagination and auth.
- [ ] SD-006 STAFF Design an IoT telemetry pipeline for 50 million devices sending readings every 10 seconds over flaky cellular links, where devices buffer data offline and replay it out of order.
- [ ] SD-007 STAFF Design a company-wide logging platform that takes in 20 PB of application logs a month from 10,000 services, with 7 days of fast search and 1 year of cheap cold storage.
- [ ] SD-008 STAFF Design an event pipeline with exactly-once delivery from a payments service to the ledger warehouse at 50,000 events a second, running across two regions.
- [ ] SD-009 SENIOR Design a schema registry and event contract process for 300 Kafka topics owned by 60 teams, so a breaking change is caught before it reaches production.
- [ ] SD-010 STAFF Design a mobile and web event SDK plus collection backend for 2 billion daily active users, with client-side batching, sampling and a kill switch for a bad event without an app release.
- [ ] SD-011 SENIOR Design a dead letter queue and replay system for a stream of 300,000 events a second, where 0.1% of events fail parsing and must be fixed and replayed within a day.

Real-time analytics, ads and experimentation Weeks 4–6

- [ ] SD-012 SENIOR Design a real-time sales dashboard for a retail chain with 3,000 stores and 20,000 checkout events a second, with numbers less than 10 seconds old.
- [ ] SD-013 STAFF Design ad impression and click aggregation for 10 billion impressions a day, where advertisers are billed from the counts and any double counting costs real money.
- [ ] SD-014 STAFF Design click fraud filtering inside an ad aggregation pipeline at 2 million events a second, where invalid clicks found up to 48 hours later must be removed from advertiser bills.
- [ ] SD-015 SENIOR Design a top-K trending hashtags service over 500,000 posts a second, with 1-minute, 1-hour and 24-hour windows.
- [ ] SD-016 STAFF Design an A/B testing platform that runs 5,000 experiments at once across 1 billion users and computes 200 metrics per experiment every day with confidence intervals.

- [ ] SD-017 STAFF Design assignment and exposure logging for an experimentation platform with 300 million users, where users switch devices and log in partway through an experiment.
- [ ] SD-018 STAFF Design a company metrics platform (a semantic layer) where 1,000 metrics are defined once and give the same answer in dashboards, notebooks, experiments and executive reports.
- [ ] SD-019 SENIOR Design a real-time count of unique viewers for 100,000 concurrent live streams, where an approximate answer within 2% error is acceptable.
- [ ] SD-020 STAFF Design a streaming join of order, payment and delivery events for a food delivery app at 50,000 events a second, where any stream can be late or missing and order status must be correct within 1 minute.
- [ ] SD-021 STAFF Design a real-time OLAP store for an internal product analytics tool where 5,000 analysts slice 100 billion rows by 40 dimensions and expect answers in under a second.

Batch processing, backfills and reprocessing Weeks 5–6

- [ ] SD-022 SENIOR Design a daily Spark job that joins a 50 TB events table with a 2 TB users table, where 1% of users create 40% of events and skew keeps crashing the job.
- [ ] SD-023 STAFF Design a backfill system that can safely recompute 2 years of daily partitions for 300 downstream tables after a bug fix, without breaking dashboards people use during the day.
- [ ] SD-024 SENIOR Design incremental processing for a 100 TB fact table that gets late updates to any date in the last 90 days, so you never rebuild the whole table.
- [ ] SD-025 STAFF Design the recovery and reprocessing plan after a bad deploy wrote wrong revenue numbers for 6 hours into both the real-time store and the warehouse.
- [ ] SD-026 STAFF Design workflow orchestration for 20,000 daily tasks across 80 teams, with SLAs, retries, cross-team dependencies and clear ownership when a task fails at 3 a.m.
- [ ] SD-027 STAFF Design a batch and streaming setup for session metrics where the real-time number and the next-day batch number must match within 0.5%.
- [ ] SD-028 SENIOR Design a sessionization job for 30 billion page views a day, where a session ends after 30 minutes of inactivity and sessions can cross midnight and time zones.

Data modeling and product warehouses Week 7

- [ ] SD-029 SENIOR Design the data model and pipeline for a hotel bookings warehouse with 2 million bookings a day, where bookings can be changed, cancelled and partly refunded weeks later.
- [ ] SD-030 SENIOR Design the warehouse for a ride-sharing app with 30 million trips a day that supports driver pay, surge pricing analysis and city-level dashboards.
- [ ] SD-031 STAFF Design a payments warehouse that reconciles 100 million transactions a day against bank settlement files, with many currencies and chargebacks that arrive up to 120 days later.
- [ ] SD-032 SENIOR Design the data model for streaming playback analytics that tracks start time, rebuffering and watch time for 200 million daily viewers across TVs, phones and browsers.

- [ ] SD-033 SENIOR Design an e-commerce warehouse for orders, returns and inventory across 20 countries, where prices and categories change over time and reports must use the value at the time of sale.
- [ ] SD-034 STAFF Design a social feed analytics warehouse that shows how ranking changes affect time spent, for 3 billion users and 100 billion feed impressions a day.
- [ ] SD-035 SENIOR Design the data model and pipeline behind a creator earnings dashboard that shows daily earnings from ads, subscriptions and tips, matching finance payouts to the cent.
- [ ] SD-036 STAFF Design the data model and pipeline behind a "Year in Review" feature that gives 500 million users a personal summary, computed in 2 weeks and launched on a single day.
- [ ] SD-037 SENIOR Design the data model and pipeline behind a seller dashboard for a marketplace with 5 million sellers, showing sales, fees and payouts updated every 15 minutes.
- [ ] SD-038 STAFF Design a customer 360 table that links web cookies, app device IDs, emails and phone numbers for 800 million people, where identity links can be added and removed later.
- [ ] SD-039 STAFF Design the data model for a subscription business that reports MRR, churn and cohort retention, where plans upgrade, downgrade, pause and get refunded mid-cycle.

Lakehouse platforms and migrations Week 8

- [ ] SD-040 STAFF Design a lakehouse platform on Iceberg for 50 PB of data and 3,000 users, covering table format, catalog, compaction, snapshot expiry and access control.
- [ ] SD-041 SENIOR Design the fix for a streaming job that writes to an Iceberg table every minute and has created 40 million small files in one month.
- [ ] SD-042 STAFF Design a migration of 30,000 Hive tables to Iceberg with zero downtime for 2,000 daily jobs and no change in query results.
- [ ] SD-043 STAFF Design the move of a 10 PB on-prem Hadoop platform to the cloud over 12 months, with both systems running in parallel and numbers that must match before cutover.
- [ ] SD-044 SENIOR Design the table layout (partitioning, clustering and sort order) for a 5 PB events table that is queried both by date range and by user ID.
- [ ] SD-045 STAFF Design a zero-downtime move from a self-managed Kafka cluster handling 3 GB a second to a managed streaming service, with no lost or duplicated events.
- [ ] SD-046 SENIOR Design upserts and deletes into a 200 TB lakehouse table from a CDC stream of 100,000 changes a second, and choose between copy-on-write and merge-on-read.
- [ ] SD-047 STAFF Design a multi-engine lakehouse where Spark, Trino, Flink and a Python notebook service all read and write the same tables safely, with one catalog and one set of permissions.

Data quality, lineage and privacy Weeks 8 and 13

- [ ] SD-048 STAFF Design a data quality platform that runs checks on 20,000 tables daily and stops bad data from reaching 500 downstream dashboards, without so many alerts that people ignore them.
- [ ] SD-049 SENIOR Design anomaly detection on row counts and key metrics for 5,000 daily pipelines, where weekends, holidays and launches look like anomalies but are not.

- [ ] SD-050 STAFF Design column-level lineage for a warehouse with 100,000 tables built from SQL, Spark and dbt, so an engineer can see every dashboard affected by a column change.
- [ ] SD-051 SENIOR Design a data catalog and search for 200,000 datasets, where a user must find the one trusted revenue table among 40 tables with similar names.
- [ ] SD-052 STAFF Design a GDPR deletion platform that removes one user's data from 3,000 tables, logs, backups and ML training sets within 30 days, with proof that it happened.
- [ ] SD-053 STAFF Design a PII detection and tagging system that scans 10 PB of existing data plus every new column each day, then applies masking and access rules automatically.
- [ ] SD-054 SENIOR Design freshness and SLA tracking for 1,000 critical tables, so a late upstream job alerts the owners of every affected downstream table.
- [ ] SD-055 STAFF Design a consent system where the privacy choices of 1 billion users (ads, analytics, AI training) are enforced in every pipeline that reads their data.
- [ ] SD-056 SENIOR Design the debugging tools and runbook for when daily revenue drops 8% overnight and you must decide within 1 hour if it is a real change or a pipeline bug.

Multi-tenant compute and cost Week 9

- [ ] SD-057 STAFF Design a multi-tenant Spark platform on Kubernetes for 150 teams sharing 20,000 cores, with fair scheduling, quotas and protection from one team's runaway job.
- [ ] SD-058 SENIOR Design autoscaling for a Flink job on Kubernetes whose traffic swings from 50,000 to 800,000 events a second during a daily peak.
- [ ] SD-059 STAFF Design a cost attribution system that shows each team its monthly compute, storage and query spend on a data platform that costs 40 million dollars a year.
- [ ] SD-060 STAFF Design a cost-aware query platform that cuts warehouse spend by 30% without slowing down the 500 most important dashboards.
- [ ] SD-061 SENIOR Design a storage tiering policy for 80 PB in S3 where 90% of reads touch the last 30 days of data but audits need 7 years.
- [ ] SD-062 STAFF Design an interactive SQL service in the style of Trino for 3,000 analysts with 400 concurrent queries at peak, where no single query can starve the rest.
- [ ] SD-063 STAFF Design cross-region disaster recovery for a warehouse and its 3,000 pipelines, with a recovery time of 4 hours and less than 15 minutes of lost data.

Feature stores and ML data Week 10

- [ ] SD-064 STAFF Design a feature store serving 2,000 features to 50 models, with online lookups in less than 10 ms at 1 million requests a second and the same values used in offline training.
- [ ] SD-065 SENIOR Design point-in-time correct training data for a churn model with 100 million users, so no feature leaks information from after the prediction date.
- [ ] SD-066 STAFF Design real-time fraud features for card payments at 20,000 transactions a second, such as spend per card and per merchant in the last 5 minutes, with a decision in less than 50 ms.
- [ ] SD-067 SENIOR Design the feature pipeline for a recommendation system that needs user embeddings refreshed daily and recent click history refreshed within 1 minute.

- [ ] SD-068 STAFF Design the training data pipeline for a feed ranking model that logs 50 billion impressions a day, with features captured at serving time and labels that arrive up to 7 days later.
- [ ] SD-069 SENIOR Design a labeling data pipeline where 5,000 human labelers tag 2 million images a week, with quality checks, consensus voting and versioned datasets.
- [ ] SD-070 STAFF Design model monitoring that detects feature drift and prediction drift for 300 production models, and links each alert to the upstream table that changed.
- [ ] SD-071 STAFF Design an embedding platform for 5 billion items that rebuilds all embeddings when the model changes and keeps old and new versions live during the switch.
- [ ] SD-072 SENIOR Design dataset versioning so any model trained in the last 2 years can be retrained on exactly the same data.

AI engineering: RAG, search and evaluation Week 11

- [ ] SD-073 STAFF Design RAG over 10 million company documents from Google Drive, Confluence and Slack, where every answer must respect the asking employee's document permissions.
- [ ] SD-074 STAFF Design the ingestion and re-indexing pipeline for a RAG system where 200,000 documents change every day and deleted documents must disappear from answers within 15 minutes.
- [ ] SD-075 STAFF Design a search indexing pipeline for a marketplace with 500 million listings, where price and stock changes must show up in search within 30 seconds.
- [ ] SD-076 STAFF Design an evaluation platform for an AI assistant that runs 50,000 test prompts on every model or prompt change and reports quality, safety and cost regressions within 2 hours.
- [ ] SD-077 SENIOR Design the logging and feedback pipeline for a chatbot with 20 million conversations a day, capturing ratings, edits and follow-up questions for later training.
- [ ] SD-078 STAFF Design an LLM-as-judge pipeline that scores 1 million model outputs a day, with a way to measure how often the judge agrees with human raters.
- [ ] SD-079 SENIOR Design hybrid keyword and vector search over 100 million support tickets, with filters by product, date and customer tier.

AI infrastructure: inference, training data and agents Week 12

- [ ] SD-080 STAFF Design an LLM inference platform serving 5 models to 300 internal teams at 100,000 requests a minute, with per-team quotas, priority tiers and token-level billing.
- [ ] SD-081 STAFF Design the pipeline that turns 5 PB of raw web crawl into a deduplicated, filtered and PII-scrubbed pre-training dataset, with every filter step reproducible.
- [ ] SD-082 STAFF Design agent infrastructure for 1 million agent runs a day that call tools and keep memory across sessions, where every step is logged so a failed run can be replayed.
- [ ] SD-083 SENIOR Design usage and token metering for an AI API with 50,000 customers, where bills must be exact and customers see their usage within 1 minute.
- [ ] SD-084 STAFF Design checkpoint storage and data loading for a 10,000-GPU training cluster, where a data loader stall wastes thousands of GPU hours.
- [ ] SD-085 SENIOR Design a GPU usage analytics pipeline that shows each research team its GPU hours, idle time and failed jobs across 20,000 GPUs, updated every 5 minutes.

Staff-only: org-level platform problems Weeks 14–15

- [ ] SD-086 STAFF Your company has 40 teams each building their own pipelines with different definitions of active user. Design the platform and the plan to fix it.
- [ ] SD-087 STAFF Leadership wants the data platform bill cut from 60 million to 40 million dollars a year within 12 months without hurting product teams. Design the plan, the tooling and how you will measure success.
- [ ] SD-088 STAFF After two acquisitions your company runs Redshift, Snowflake and BigQuery side by side. Design the target architecture and a 2-year consolidation plan.
- [ ] SD-089 STAFF Data incidents reach executives twice a month and nobody knows who owns the broken tables. Design the ownership model, tooling and on-call process that fixes this.
- [ ] SD-090 STAFF The company launches an AI assistant in 6 months and 15 product teams need shared data for RAG, fine-tuning and evaluation. Design the platform and the roadmap.
- [ ] SD-091 STAFF A new privacy law means you must show where every piece of personal data lives and who used it, across 8,000 pipelines, within 9 months. Design the system and the rollout.
- [ ] SD-092 STAFF Your 25 ML teams have each built their own feature pipelines and training data jobs with heavy duplication. Design a shared ML data platform and how you will get teams to adopt it.
- [ ] SD-093 STAFF The batch warehouse is 24 hours behind and product teams now want everything in real time. Design the strategy, including what should and should not become real time.
- [ ] SD-094 STAFF A data platform team of 30 engineers supports 2,000 internal users and is drowning in support tickets. Design a self-serve platform and a plan to cut tickets in half.
- [ ] SD-095 STAFF Your company runs 400 streaming jobs with no shared standards, and last month one bad job took down a shared Kafka cluster. Design the guardrails, the platform and the migration plan.
- [ ] SD-096 STAFF Your product will let 10,000 enterprise customers bring their own data for analytics and AI features, with strict tenant isolation. Design the platform.
- [ ] SD-097 STAFF The in-house pipeline framework is 8 years old, runs 70% of all jobs and only 2 people understand it. Design the plan to replace or modernize it without freezing new work.
- [ ] SD-098 STAFF The company is expanding into 5 new countries with data residency rules. Design the data architecture so a new region can be added in weeks, not quarters.
- [ ] SD-099 STAFF Researchers, analysts and product engineers want different things from the data platform and keep escalating to leadership. Design the operating model, platform tiers and roadmap.
- [ ] SD-100 STAFF You are the first staff data engineer at a 200-person AI startup with no data platform. Design what you build in the first 6, 12 and 24 months and what you choose not to build.

TOPIC 11 OF 11

Behavioral and leadership

100 questions · 40 Senior · 60 Staff

Influence and leading without authority

Weeks 3–15, two a week

- [ ] BH-001 SENIOR Tell me about a time you convinced another team to change their roadmap to support your project, without any authority over them.
- [ ] BH-002 STAFF Tell me about a time you got five or more teams to adopt a standard, tool or platform they did not ask for. How did you win over the first team?
- [ ] BH-003 SENIOR Describe a time you changed a more senior engineer's mind on a technical decision. What evidence did you bring?
- [ ] BH-004 STAFF Tell me about a time you built support for a big proposal before the formal review, so the decision was mostly made before the meeting.
- [ ] BH-005 STAFF Tell me about a time you had to influence a director or VP outside your management chain. What did you do differently for that audience?
- [ ] BH-006 SENIOR Tell me about a time your idea was rejected at first. What did you change, and did it happen in the end?
- [ ] BH-007 STAFF Describe a time you led engineers from several teams, none of them reporting to you, toward a shared goal. How did you keep them committed?
- [ ] BH-008 STAFF Tell me about a time a design doc or strategy doc you wrote changed how a whole org worked.

Technical direction, strategy and cross-team alignment

Weeks 3–15, two a week

- [ ] BH-009 STAFF Tell me about a time you set the technical direction for a data platform for the next 2 to 3 years. How did you decide what to bet on?
- [ ] BH-010 STAFF Walk me through a technical strategy you wrote. Who read it, what changed because of it, and what did you get wrong?
- [ ] BH-011 SENIOR Tell me about a project you led that needed work from three or more teams. How did you keep the plan in sync?
- [ ] BH-012 STAFF Tell me about a time two teams were building overlapping systems. How did you find out and what did you do?
- [ ] BH-013 STAFF Describe a time you got engineering, product and data science to agree on one definition of a key metric.
- [ ] BH-014 SENIOR Tell me about a time you picked a new technology for your team. How did you evaluate it and limit the risk?
- [ ] BH-015 STAFF Tell me about a time you stopped a popular project or technology choice because you believed it was the wrong direction.
- [ ] BH-016 STAFF Tell me about a cross-team project that slipped badly. How did you get it back on track, and what did you change for next time?

Conflict and disagree-and-commit Weeks 3–15, two a week

- [ ] BH-017 SENIOR Tell me about a time you strongly disagreed with your manager. How did you raise it and what happened?
- [ ] BH-018 SENIOR Tell me about a conflict with a peer engineer over a design. How did you resolve it?
- [ ] BH-019 STAFF Tell me about a conflict between your team and another team over who owns a system or dataset. How was it settled?
- [ ] BH-020 SENIOR Tell me about a time a product manager kept changing requirements late in a project. How did you handle it?
- [ ] BH-021 STAFF Tell me about a time a senior stakeholder asked for a number or dashboard you knew would mislead people. What did you do?
- [ ] BH-022 SENIOR Tell me about a time you disagreed with a decision but committed to it fully. How did you show that commitment?
- [ ] BH-023 STAFF Tell me about a time you committed to a decision you disagreed with and it later proved wrong. What did you do then?
- [ ] BH-024 STAFF Tell me about a time you had to settle a conflict between two strong engineers on different teams.
- [ ] BH-025 SENIOR Tell me about a time you were the one who was wrong in a disagreement. How did you realize it and what did you do next?

Failure, mistakes, incidents and on-call Weeks 3–15, two a week

- [ ] BH-026 SENIOR Tell me about your biggest technical mistake. What was the impact and what did you change afterwards?
- [ ] BH-027 SENIOR Tell me about a time bad data reached a dashboard, model or customer. How did you find out and what did you do in the first hour?
- [ ] BH-028 STAFF Tell me about the worst production incident you led. How did you coordinate people, update leadership and decide when to roll back?
- [ ] BH-029 STAFF Tell me about a postmortem you ran where the root cause was another team's change. How did you keep it blameless and still get fixes done?
- [ ] BH-030 SENIOR Tell me about a time you were on call and had to make a hard call at night with very little information.
- [ ] BH-031 STAFF Tell me about a time you cut the on-call load or incident rate for a whole org. What were the numbers before and after?
- [ ] BH-032 STAFF Tell me about a project you led that failed or was cancelled. What would you do differently?
- [ ] BH-033 SENIOR Tell me about a time you missed something in a code or design review that later caused a problem.

Ambiguity, prioritization and trade-offs Weeks 3–15, two a week

- [ ] BH-034 SENIOR Tell me about a time you were given a vague goal with no clear requirements. How did you turn it into a plan?

☐	BH-035	STAFF	Tell me about a time nobody owned an important system or dataset. How did you find an owner or take it on yourself?
☐	BH-036	STAFF	Tell me about a time you had three important projects and people for only one. How did you decide, and who did you tell?
☐	BH-037	SENIOR	Tell me about a time you said no to a request from a senior stakeholder. How did you say it?
☐	BH-038	STAFF	Tell me about a time you paused or killed your own team's project so a more important one could move faster.
☐	BH-039	SENIOR	Tell me about a technical decision you made with incomplete data. How did you limit the damage if you were wrong?
☐	BH-040	STAFF	Tell me about a build-versus-buy decision for a data platform component. What drove the decision, and was it right?
☐	BH-041	STAFF	Tell me about a time you chose speed of delivery over long-term quality. How did you manage the cost later?
☐	BH-042	SENIOR	Tell me about a time the problem you were asked to solve turned out to be the wrong problem.
☐	BH-043	STAFF	Tell me about a time priorities changed in the middle of your project. How did you decide what to keep, cut or hand off?
☐	BH-044	STAFF	Tell me about a decision you made that was very hard to reverse. How did you decide it was worth it, and who did you involve?

Simplification, raising the bar and growing others

Weeks 3–15, two a week

☐	BH-045	STAFF	Tell me about a time you simplified a complex system that many teams depended on. What did you remove and how did you prove it was safe?
☐	BH-046	SENIOR	Tell me about a time you paid down tech debt. How did you get time for it and how did you measure the result?
☐	BH-047	STAFF	Tell me about a time you made the case to leadership for a large tech debt or reliability investment. What numbers did you use?
☐	BH-048	SENIOR	Tell me about a time you pushed back in a code review that others had already approved. What was the issue and how did the author react?
☐	BH-049	STAFF	Tell me about a standard you introduced, such as testing, data contracts or design reviews, that raised the quality bar across several teams.
☐	BH-050	STAFF	Tell me about a time you argued against hiring someone others liked, or for someone others doubted. What happened?
☐	BH-051	SENIOR	Tell me about someone you mentored. What did they struggle with, what did you do, and where are they now?
☐	BH-052	STAFF	Tell me about a time you helped a senior engineer grow into a tech lead or staff role. What did you deliberately hand over to them?
☐	BH-053	SENIOR	Tell me about a time you gave hard feedback to a peer. How did they take it?
☐	BH-054	STAFF	Tell me about a time you raised the technical level of a whole team, not just one person.

Delivery, business impact and data-driven decisions Weeks 3–15, two a week

- [ ] BH-055 SENIOR Tell me about a time you delivered a project on a tight deadline. What did you cut and how did you decide?
- [ ] BH-056 STAFF Tell me about the project with the biggest business impact in your career. Give the numbers in revenue, cost or users.
- [ ] BH-057 SENIOR Tell me about a time you cut the cost or runtime of a data system. What were the before and after numbers, and how did you measure them?
- [ ] BH-058 STAFF Tell me about a time data you produced changed a product or business decision. What was the decision and what happened?
- [ ] BH-059 SENIOR Tell me about a time the data told a different story from what leadership believed. How did you present it?
- [ ] BH-060 STAFF Tell me about a time you set success metrics for a platform before building it. Did you hit them?
- [ ] BH-061 SENIOR Tell me about a time you saw a project was going to miss its deadline. When did you tell people and what options did you give them?

Ethics, privacy, security and research partnerships Weeks 3–15, two a week

- [ ] BH-062 SENIOR Tell me about a time you found sensitive data where it should not be, such as personal data in logs. What did you do?
- [ ] BH-063 STAFF Tell me about a time you pushed back on a use of data that was legal but felt wrong for users.
- [ ] BH-064 STAFF Tell me about a time you balanced a security or privacy requirement against a team's need to move fast.
- [ ] BH-065 STAFF Tell me about a time you worked closely with researchers or scientists. How did you handle their changing needs and different way of working?
- [ ] BH-066 SENIOR Tell me about a time a research or ML team needed data infrastructure that did not exist yet. How did you decide what to build first?
- [ ] BH-067 STAFF Tell me about a time you had to explain an infrastructure limit to a research team that wanted something faster or bigger. How did you find a way forward?

Motivation and career goals Weeks 14–15

- [ ] BH-068 SENIOR Why do you want a staff role, and what do you think changes day to day from senior to staff?
- [ ] BH-069 SENIOR Why this company, and why now? What do you know about its data and AI infrastructure problems?
- [ ] BH-070 SENIOR Why are you leaving your current job?
- [ ] BH-071 SENIOR Where do you want to be in 5 years, on a deep technical track or in management, and why?
- [ ] BH-072 STAFF What is the largest scope you have owned so far, and what larger scope do you want to own in this role?

Deep dive into your most complex project Weeks 14–15

- [ ] BH-073 STAFF Walk me through the most complex data system you have designed or led. Start with the problem and the scale.
- [ ] BH-074 STAFF On that project, why did you choose your design over the two strongest alternatives?
- [ ] BH-075 STAFF If you rebuilt that system today, what would you change and why?
- [ ] BH-076 SENIOR What were the real numbers on that project: data volume, latency, cost, number of users and team size?
- [ ] BH-077 STAFF Who disagreed with your approach on that project, and how did you resolve it?
- [ ] BH-078 SENIOR What exactly did you personally do on that project, as opposed to your team?
- [ ] BH-079 STAFF What broke in production after that system launched, and what did it teach you about the design?
- [ ] BH-080 STAFF How did that project change the way other teams worked, and how do you know?

Leadership principles style round Week 15

- [ ] BH-081 SENIOR Tell me about a time you dug into raw data or code and found that a summary everyone trusted was wrong.
- [ ] BH-082 STAFF Tell me about a time you started from what the customer needed and worked backwards, and it changed the technical plan.
- [ ] BH-083 SENIOR Tell me about a time you built a small tool or automation that saved other engineers hours every week.
- [ ] BH-084 STAFF Tell me about a time you actively looked for opinions that challenged your plan before a big decision. What did you learn that changed it?
- [ ] BH-085 SENIOR Tell me about a time you acted quickly without all the information because waiting would have cost more.
- [ ] BH-086 STAFF Tell me about a time you delivered more with fewer people or less budget than you asked for.
- [ ] BH-087 STAFF Tell me about a time you thought much bigger than the original ask. Did it pay off?
- [ ] BH-088 SENIOR Tell me about a time you had to learn a new area fast because a project needed it.
- [ ] BH-089 STAFF Tell me about a time you refused to lower the quality bar even though it delayed a launch.
- [ ] BH-090 STAFF Tell me about a time you lost the trust of a team or stakeholder. How did you earn it back?

Ownership-style round Week 15

- [ ] BH-091 SENIOR Tell me about a time you fixed a problem that was not your job because nobody else was going to.
- [ ] BH-092 STAFF Tell me about a system you owned end to end for more than a year. How did you decide what to improve each quarter?
- [ ] BH-093 STAFF Tell me about a time you thought about the 3-year cost of a design, not just the launch. What did you do differently because of it?

☐	BH-094	STAFF	Tell me about a time you took over a failing project you did not start. What did you do in the first 30 days?
☐	BH-095	SENIOR	Tell me about a time you spotted a risk long before anyone else and acted before it became an incident.
☐	BH-096	STAFF	Tell me about a time you stayed accountable for an outcome, such as adoption or reliability, after the project officially ended.
☐	BH-097	STAFF	Tell me about a time you found a system you built being used in a way you never intended. What did you do?
☐	BH-098	STAFF	Tell me about a decision you owned that went badly in front of leadership. What did you say and do?
☐	BH-099	STAFF	Tell me about a time you chose an unglamorous project that mattered more over a high-visibility one.
☐	BH-100	STAFF	Tell me about a commitment you made to another team on behalf of your team. How did you keep it when things got hard?

BONUS

Reported in real interviews

95 questions from public write-ups, 2024–2026 · each links to its source

- Each question comes from one person's write-up on Blind, LeetCode Discuss, Reddit, YouTube or an interview-prep site. Treat it as a sample of what gets asked, not a list of what will be asked.
- Wording is quoted or closely paraphrased. Few write-ups come from staff loops; where one does, the line says so.
- Sections marked **Not in the plan yet** are rounds the 15 weeks don't train.

Coding: Python and DSA

Weeks 1–2

- [ ] RQ-01 A bookstore has customer comments from each shop location. Find the most common comment across all locations, ignoring duplicates from the same location.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-02 Given a list of workshops, return the largest number of classes hosted in total across any run of consecutive years that each had at least one workshop.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-03 Given meetings with a start time, an end time and a number of people, find the most people that were in a meeting at once.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-04 Given an array of book prices and a budget, find the maximum number of unique books you can buy.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-05 Given employee, department and salary data stored in a dictionary, find the second-highest salary in each department.
  Meta · Senior DE screen · 2026 · Aced
- [ ] RQ-06 Write a function that reads and processes a CSV file, and handles the case where the file is not found.
  Meta · Senior DE screen · 2026 · Aced
- [ ] RQ-07 Given a stream of Uber ride requests, count the requests in every 15–minute tumbling window.
  Meta · Senior DE onsite · 2026 · Aced
- [ ] RQ-08 Given a dictionary of users and their values, find the values that only the current user has and no other user has.
  Meta · Senior+ DE screen (IC6 target) · Sep 2026 · PracHub
- [ ] RQ-09 Given user view times as (start, end) tuples that may overlap, find the unique time ranges viewed.
  Netflix · L5 DE screen · Dec 2024 · Blind
- [ ] RQ-10 Take a single-threaded solution, turn it into a Spark job, then into a real-time streaming version, and explain the trade-offs at each step.
  Netflix · Senior DE (L5) · Oct 2025 · LeetCode
- [ ] RQ-11 Traverse a hierarchy in Python (depth-first search).
  Airbnb · Senior DE · Oct 2024 · Blind
- [ ] RQ-12 Course Schedule I and II (topological sort), and a problem combining merge intervals with binary search.
  Uber · DE (SDE 2) · Jan 2024 · LeetCode

- [ ] RQ-13 Using a users_viewership table, design an algorithm for the view count of each movie (merge-intervals logic).
  Uber · L4 DE · Apr 2024 · LeetCode
- [ ] RQ-14 Solve a hard LeetCode problem (BFS plus dynamic programming) in Python in 25 minutes.
  DoorDash · Staff DE screen · Feb 2024 · LeetCode
- [ ] RQ-15 House Robber II and binary tree serialize/deserialize. Solve each in SQL first, then write the equivalent Python.
  LinkedIn · Staff DE · Jan 2024 · LeetCode
- [ ] RQ-16 Merge two sorted arrays. Then: find the smallest missing positive integer.
  Amazon · DE II phone screen · Dec 2024 · LeetCode
- [ ] RQ-17 A variant of merge intervals. Follow-up: how would you solve it at scale?
  Google · SWE L6 (staff) · Mar 2025 · LeetCode
- [ ] RQ-18 Compute shipping cost with tiered pricing. Then: compute transaction fees from a CSV string.
  Stripe · DE screen · 2025–2026 · PracHub

Coding: concurrency and low-level design Not in the plan yet

- [ ] RQ-19 Build a web crawler. Follow-up: make it multithreaded, using thread pool executors and asyncio.
  Anthropic · ML infra phone screen · Jan 2026 · LeetCode
- [ ] RQ-20 Build an encoder and decoder using RLE and BP.
  Databricks · Senior SWE · Apr 2026 · LeetCode
- [ ] RQ-21 Write an iterator for a set that can return a snapshot of the set at any point in time.
  Databricks · Senior SWE · Apr 2026 · LeetCode
- [ ] RQ-22 Design a log writer that is multi-threaded but keeps the writes in order.
  Databricks · Senior SWE (low-level design) · Apr 2026 · LeetCode
- [ ] RQ-23 In a distributed N-ary tree where nodes talk only by async messages, count the nodes and print the result at the root.
  Snowflake · Backend SWE · Jun 2026 · LeetCode

SQL Weeks 1–2

- [ ] RQ-24 Calculate the percentage of total sales completed on the same day the customer registered.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-25 Find customers who bought 3 or more books on both the first and last day of sales, excluding those with only one transaction.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-26 Find the top 5 customers, ordered by the average payment per book made by the people they invited.
  Meta · DE screen · Apr 2025 · Blind
- [ ] RQ-27 Follow-up on your answer: how would you write it without a CTE?
  Meta · DE onsite · Apr 2025 · Blind
- [ ] RQ-28 Find the number of people who checked out a book on the same day that another person returned that same book.
  Meta · Senior DE screen · 2026 · Aced

- [ ] RQ-29 Car rental schema: daily rental utilisation by city and car size, compared against inventory.
  Meta · DE onsite · Dec 2025 · PracHub
- [ ] RQ-30 Rewrite your query to minimise table scans.
  Meta · DE onsite · Sep 2025 · PracHub
- [ ] RQ-31 How long did each user stay on a page? (LEAD, then a difference.)
  Airbnb · DE onsite · Sep 2026 · PracHub
- [ ] RQ-32 Find super users: users with impressions on 7 consecutive days. Then: top N, and how do you handle ties?
  Airbnb · DE onsite · Sep 2026 · PracHub
- [ ] RQ-33 Find super hosts: at least three listings, an average rating above a threshold, and at least 90% of listings with a review in the last 30 days.
  Airbnb · DE · Sep 2026 · PracHub
- [ ] RQ-34 Find the median price for each city.
  Airbnb · DE online assessment · Sep 2026 · PracHub
- [ ] RQ-35 Find drivers who haven't taken a trip in the first 7 days after signup. Then: how would you optimise it as the data grows?
  Uber · L4 DE · Apr 2024 · LeetCode
- [ ] RQ-36 Find the top 3 cities each month.
  Uber · L4 DE · Apr 2024 · LeetCode
- [ ] RQ-37 Find the total time each driver spent driving each day, when a trip's start and end can fall on different days.
  Uber · L4 DE · Apr 2024 · LeetCode
- [ ] RQ-38 A hard SQL problem with window functions. Then: how would you solve it without window functions?
  Uber · DE screen · Feb 2025 · YouTube
- [ ] RQ-39 Find users who changed their city more than 3 times in the last year.
  Amazon · DE II phone screen · Dec 2024 · LeetCode
- [ ] RQ-40 Print the distinct pairs of connected cities, where a → b and b → a count once.
  Amazon · DE II phone screen · Dec 2024 · LeetCode
- [ ] RQ-41 Implement row_number() without using window functions.
  Amazon · DE · May 2024 · LeetCode
- [ ] RQ-42 Live SQL: find users with consecutive logins.
  Amazon · DE · Jul 2026 · LeetCode

Data modeling Weeks 7–8

- [ ] RQ-43 Netflix is launching an ad-supported plan and your team builds the ad system. Design the data models that let ad operations and business teams traffic direct-sold demand into the system.
  Netflix · Senior SWE (data modeling round) · Oct 2025 · LeetCode
- [ ] RQ-44 Design a model for cumulative booking metrics that also shows first and last booking times. Then defend every design choice, and optimise the queries and tables.
  Airbnb · DE · Sep 2026 · PracHub
- [ ] RQ-45 Reels case: model the entities for feed content and shares.
  Meta · DE onsite · Dec 2025 · PracHub

- [ ] RQ-46 Follow-ups on your model: what if I want to track changes to this? Is it one-to-many or many-to-many? How do you handle a many-to-many relationship?
  Meta · DE onsite · Sep 2025 · PracHub

- [ ] RQ-47 Design a data model for an Uber ride app. How would you optimise the transactional table once it grows beyond 1 TB? Round-robin or hash partitioning?
  Meta · Senior DE onsite · 2026 · Aced

- [ ] RQ-48 Extend the model to answer new business questions. Design physical tables that handle big data at scale, then optimise them for the key analytical questions.
  Meta · L7 DE screen (recruiter agenda) · Feb 2025 · LeetCode

- [ ] RQ-49 Build a data model for an app you use every day, then answer a set of SQL queries on it. (Graded on whether the queries are easy or complex on your model; bridge tables and SCD came up.)
  Uber · DE · Feb 2025 · YouTube

- [ ] RQ-50 For a top-10-movies report, design the remaining source tables and the warehouse tables (facts and dimensions).
  Uber · L4 DE · Apr 2024 · LeetCode

- [ ] RQ-51 An orders and transactions schema with facts and dimensions, handling Prime status as a slowly changing dimension. A clickstream and session schema, with partitions and sort keys.
  Amazon · DE (candidate's prep list) · Sep 2025 · Blind

Pipeline and system design Weeks 3–8 and 14–15

- [ ] RQ-52 Design an experimentation (A/B testing) platform, and build the data pipeline for it.
  LinkedIn · Staff DE · Jan 2024 · LeetCode

- [ ] RQ-53 Design a key-value store.
  LinkedIn · Staff DE · Jan 2024 · LeetCode

- [ ] RQ-54 Design config rollout and click aggregation.
  Netflix · DE onsite · Dec 2025 · PracHub

- [ ] RQ-55 Design a pipeline for Netflix clickstream events, feeding an hourly dashboard of the top trending movies in each location.
  Uber · L4 DE · Apr 2024 · LeetCode

- [ ] RQ-56 Design the backend of a near-real-time dashboard that shows the trending dishes in a city.
  Uber · DE (SDE 2) · Jan 2024 · LeetCode

- [ ] RQ-57 Design a system that returns the top 10 products.
  Amazon · DE II · Dec 2024 · LeetCode

- [ ] RQ-58 ETL design: how do you handle messy or late data, retries and idempotency? When do you choose streaming over batch?
  Amazon · DE · Sep 2025 · Blind

- [ ] RQ-59 Walk through the steps data takes from capture to the end, and how you'd optimise the queries and tables along the way.
  Meta · Senior+ DE screen (IC6 target) · Sep 2026 · PracHub

- [ ] RQ-60 Build a payment system for an online game platform, covering refunds, credits and in-game items. Then: concurrent transactions, and how failure recovery works.
  Databricks · Senior SWE · Apr 2026 · LeetCode

- [ ] RQ-61 Here is a design on the screen. Walk the end-to-end flow, critique it, and tell us what you would change.
  Anthropic · EM, Data Infrastructure · 2026 · Aced
- [ ] RQ-62 Infra-heavy design with the maths done out loud: throughput, latency, data size, TPS.
  Anthropic · infra/platform, staff-level candidate · Feb 2026 · Blind
- [ ] RQ-63 Design an S3-like object store. Design a distributed job scheduler like Airflow. Design a messaging system like Kafka.
  Amazon, Pinterest, Airbnb · senior/staff loops · Sep 2026 · Blind
- [ ] RQ-64 Design a distributed task scheduler.
  LinkedIn · Senior DE · Sep 2024 · LeetCode
- [ ] RQ-65 Design a system that ingests data for both product and warehousing uses, and explain how you make it resilient. Then: describe a RAG pipeline.
  Unnamed AI-adjacent company · Senior DE · Apr 2026 · Reddit

Product sense and metrics Not in the plan yet

- [ ] RQ-66 How would you evaluate short video's impact on the news feed? (Users, content, interactions.)
  Meta · DE onsite · Dec 2025 · PracHub
- [ ] RQ-67 How would you measure monetization for a gaming company's product, what reports would you build, and how would you verify that a broken feature was actually fixed?
  Meta · Senior+ DE screen (IC6 target) · Sep 2026 · PracHub
- [ ] RQ-68 A metric drops on the dashboard of an Instagram-like product. What do you do? Should this metric be on the dashboard while you do the root-cause analysis?
  Meta · DE · 2025 · Aced
- [ ] RQ-69 Define the success metrics for a project (Lyft, Twitter or video theme).
  Meta · DE onsite · Aug 2024 · LeetCode
- [ ] RQ-70 Define the KPIs you'd track for a ride-share algorithm.
  Meta · IC4 DE · Nov 2024 · Blind
- [ ] RQ-71 Define a success metric. The metric drops: find the root cause. What is the leading metric? What segments would you look at?
  Meta · DE onsite · Sep 2025 · PracHub
- [ ] RQ-72 Design visualizations for streaming metrics.
  Meta · DE · Aug 2025 · PracHub
- [ ] RQ-73 Product sense in the phone screen: pick the metrics, then design the ETL (facts and dimensions) behind them.
  Netflix · Senior DE (L5) · Oct 2025 · LeetCode
- [ ] RQ-74 Describe building and improving a dashboard.
  Google · DE screen · Mar 2025 · PracHub

Project deep dive Start in week 1

- [ ] RQ-75 Walk me through your most impactful project. (Manager round, with a domain deep dive and a Spark deep dive.)
  Netflix · L5 DE · Dec 2024 · Blind
- [ ] RQ-76 Give a presentation of one of your previous projects (domain deep dive).
  Databricks · Senior Staff (L7) · May 2025 · Blind

- [ ] RQ-77 Present a recent large project end to end. How many people were involved? Can we go deeper on the GDPR data management part?
  Anthropic · EM, Data Infrastructure · 2026 · Aced
- [ ] RQ-78 Go deeper on each of the last three companies on your resume. What changed because of your work?
  Anthropic · EM, Data Infrastructure (phone) · 2026 · Aced
- [ ] RQ-79 Talk through your technical projects: their scale, complexity, and the decisions and trade-offs you made.
  Databricks · Senior SWE (behavioral) · Nov 2025 · YouTube
- [ ] RQ-80 Walk through the details of an end-to-end project.
  Airbnb · DE (behavioral) · Sep 2026 · PracHub
- [ ] RQ-81 Tell me about the best projects you've worked on, or the achievements you're proudest of.
  Uber · DE (collaboration and leadership) · Feb 2025 · YouTube

Behavioral and leadership Start in week 1

- [ ] RQ-82 Explain the impact of your project, how you mentor, and how you split work and assign credit on a team. (The candidate was rejected because the impact 'wasn't big enough'.)
  Meta · Senior+ DE leadership round (IC6 target) · Sep 2026 · PracHub
- [ ] RQ-83 Tell me about a time you had a conflict and made a data-driven decision.
  Meta · DE ownership round · Aug 2024 · LeetCode
- [ ] RQ-84 Tell me about a conflict with your team lead or manager. A process you improved that had business impact. A time you learned a tool quickly and delivered.
  Meta · DE · 2025 · Aced
- [ ] RQ-85 Tell me about a project where a stakeholder changed priority after work had started. What would you do about a feature that is profitable for the company but clearly costs customers more money? Tell me about a bug or mistake you shipped.
  Anthropic · EM, Data Infrastructure · 2026 · Aced
- [ ] RQ-86 What is your view on AI safety? (Recruiter call: 'develop a strong opinion'.)
  Anthropic · ML infra · Jan 2026 · LeetCode
- [ ] RQ-87 Tell me about a time you led a difficult project. How did you handle conflict with a teammate? Which part of your work are you most proud of?
  Databricks · hiring manager round · Nov 2025 · YouTube
- [ ] RQ-88 Leadership Principles in every screen. Prepare Dive Deep (debugging a critical pipeline under pressure) and Insist on the Highest Standards (raising data quality when deadlines were tight).
  Amazon · DE (offer) · Sep 2025 · Blind
- [ ] RQ-89 Talk about projects where you involved many teams or got leadership buy-in, and how you led and mentored others.
  Airbnb and others · Senior/Staff DE loops · Dec 2025 · Blind

Debugging and Spark depth Weeks 5–6 and 13

- [ ] RQ-90 How will Spark behave when you join an X GB dataset with a Y GB dataset and don't have enough resources? What can go wrong, and how would you optimise it?
  Uber · DE screen · Feb 2025 · YouTube

- [ ] RQ-91 Explain the physical join types: hash join and broadcast nested loop join. Then: idempotency, and change data capture (CDC).
  Amazon · DE · Jul 2026 · LeetCode
- [ ] RQ-92 Design pipelines, then debug scenarios on them.
  Netflix · Senior DE (L5) · Oct 2025 · LeetCode
- [ ] RQ-93 Query and table optimisation questions, asked without context.
  Airbnb · DE (data modeling round) · Sep 2026 · PracHub
- [ ] RQ-94 Unit and integration testing for ETL, performance tuning, and speed versus memory trade-offs in big data processing.
  Amazon · DE onsite (recruiter prep call) · Jan 2025 · Reddit
- [ ] RQ-95 An AI debugging round, alongside full-stack and behavioral rounds.
  Meta · level 6 DE onsite (one comment) · Jul 2026 · Blind